

João Pedro Coelho Encarnação

Modelo Articulado de uma Árvore Brônquica



Universidade do Algarve
Instituto Superior de Engenharia

2025/2026

João Pedro Coelho Encarnação

Modelo Articulado de uma Árvore Brônquica

Relatório de Projeto Final de Licenciatura
Licenciatura em Engenharia de Eletrotecnia e Computadores

Trabalho efetuado sob orientação do:
Professor Jorge Semião



Universidade do Algarve
Instituto Superior de Engenharia

2025/2026

Dedicatória

A preencher na versão final.

Agradecimentos

A preencher na versão final.

Resumo

O presente relatório descreve o desenvolvimento de um modelo articulado de uma árvore brônquica para apoio à simulação e treino de videobroncofibroscopia. O trabalho parte da necessidade de criar uma plataforma física de treino que permita representar, de forma controlada e realista, movimentos internos observáveis durante o procedimento, combinando técnicas como:

- impressão 3D;
- atuação mecânica;
- controlo eletrónico;
- software embarcado;
- interface homem-máquina.

A solução foi organizada em duas secções:

- Modelo dinâmico → constituído pela árvore brônquica impressa em 3D, pelos manipuladores delta, pelos servo-motores e pelo microcontrolador ESP32-S3;
- HMI → alojada num Raspberry Pi 4B, responsável por permitir ao utilizador controlar o sistema de forma simples e intuitiva, sem necessidade de memorizar comandos internos ou valores.

No modelo dinâmico foram implementados alguns perfis de movimento tais como:

- Respiração;
- Batimento cardíaco;
- Tosse.

Toda a implementação recorreu a estruturas de trajetórias, máquinas de estados, comunicação série e Equações de Cinemática Inversa para converter posições desejadas nos ângulos dos manipuladores. A interface gráfica por sua vez, permite selecionar modos de funcionamento, bem como estados de funcionamento (iniciar ou interromper o ensaio), outras funcionalidades como: estabelecer e acompanhar a comunicação série com o ESP32 e

preparar os parâmetros de movimento também se encontram implementadas, dando uma grande quantidade de ferramentas para os operadores. Durante todo o desenvolvimento foram validados, testados e modificados os varios blocos fundamentais dos sistemas já mencionados em ambos, modelo dinamico e HMI. Identificando-se no final as principais limitações do protótipo atual e possíveis sugeseões de melhorias a considerar no futuro.

Abstract

Índice Geral

	Página
Lista de Abreviaturas e Termos	xi
Lista de Figuras	xiv
Lista de Tabelas	xvi
1 Introdução	1
1.1 Enquadramento	1
1.2 Motivação	2
1.3 Objetivos	2
1.4 Metodologia	3
1.5 Organização do relatório	3
2 Estado da Arte	4
2.1 Broncoscopia e treino por simulação	4
2.2 Fidelidade dos simuladores	5
2.3 Simuladores de broncoscopia	5
2.4 Impressão 3D aplicada à simulação médica	6
2.4.1 Aplicação em simuladores de árvores brônquicas	7
2.5 Movimentos observáveis durante a broncoscopia	7
2.5.1 Movimento respiratório	8
2.5.2 Movimento cardíaco	9
2.5.3 Tosse e perturbações rápidas	9
2.5.4 Interação com o broncoscópio	10
2.5.5 Patologias e deformações locais	10
2.6 Síntese crítica	11
3 Desenvolvimento do Hardware	13
3.1 Requisitos do sistema	13
3.2 Modelo físico da árvore brônquica	14
3.2.1 Técnicas de impressão 3D	14

3.2.2	O modelo STL da árvore brônquica	15
3.2.3	Programas utilizados no tratamento do modelo	15
3.2.4	Seleção de materiais	17
3.2.5	Disposição do modelo para impressão	18
3.2.6	Evolução do processo de impressão	21
3.3	Seleção do sistema de atuação	23
3.4	Eletrônica de controlo e alimentação	25
3.4.1	Testes iniciais de eletrónica	26
3.4.2	Arquitetura de alimentação e avaliação de motores de passo .	27
3.4.3	Dimensionamento da fonte de alimentação	27
3.4.4	Gestão de energia do Raspberry Pi	28
3.4.5	Da protoboard à PCB	29
3.5	Integração mecânica e segurança	29
4	Desenvolvimentos do Software e Configurações	32
4.1	Arquitetura geral do software e ambiente de desenvolvimento	32
4.2	Movimentos representados no protótipo	34
4.3	Geração de trajetórias (Python/Raspberry Pi)	36
4.4	Estruturas de dados	40
4.5	Máquinas de estados e execução dos movimentos	40
4.6	Cinemática inversa do manipulador delta	47
4.7	Comunicação série	51
4.7.1	Comunicação série no firmware (ESP32)	51
4.7.2	Comunicação série no Python (Raspberry Pi)	55
4.8	Interface HMI no Raspberry Pi	55
4.9	Preparação e sincronização da tosse	57
4.10	Depuração e validação interna	57
5	Testes e Análise de Resultados	59
	Nota sobre o estado atual do capítulo	59
5.1	Plano de testes	59
5.2	Testes do modelo físico	59
5.3	Testes dos atuadores e da estrutura delta	60
5.4	Testes de alimentação	60
5.5	Testes de software	60
5.6	Testes de comunicação série	61
5.7	Testes de integração dos movimentos	61
5.8	Critérios de aceitação	61
5.9	Informação experimental em falta	62
6	Conclusão	64

6.1	Síntese do trabalho	64
6.2	Principais contributos	64
6.3	Limitações	65
6.4	Trabalho futuro	65
Bibliografia		67
A	Anexos	73
A.1	Anexo A: Desenhos técnicos e modelo 3D	73
A.2	Anexo B: Lista de materiais	73
A.3	Anexo C: Datasheets	73
A.4	Anexo D: Código e configurações	73
A.5	Anexo E: Protocolos de teste	73

Lista de Abreviaturas e Termos

CA Corrente Alternada; tipo de alimentação elétrica ligada à entrada da fonte de alimentação do sistema.

CAD *Computer-Aided Design*; desenho assistido por computador, usado no tratamento do modelo 3D da árvore brônquica.

CHUA Centro Hospitalar Universitário do Algarve; instituição parceira que forneceu os dados clínicos e anatômicos usados no projeto.

DEV *Development*; modo de utilização da HMI que dá acesso a ferramentas de depuração e de envio manual de comandos.

DTR *Data Terminal Ready*; sinal de controlo da porta série que pode provocar o reset do ESP32-S3.

EEPROM *Electrically Erasable Programmable Read-Only Memory*; memória não volátil usada para guardar configurações persistentes do firmware.

ESP32-S3 Microcontrolador usado para executar o firmware, interpretar comandos e gerar os sinais de controlo.

FDM *Fused Deposition Modeling*; tecnologia de impressão 3D por extrusão de filamento termoplástico, usada na maioria das peças do projeto.

FSM *Finite State Machine*; máquina de estados finitos usada para organizar a lógica de movimento, leitura série e interpretação de comandos.

GPIO *General Purpose Input/Output*; pinos digitais usados para ligação a periféricos.

HAT *Hardware Attached on Top*; placa de expansão para Raspberry Pi, acoplada aos pinos GPIO para adicionar ferramentas, periféricos ou capacidade de processamento.

HMI *Human Machine Interface*; interface homem-máquina usada para permitir ao utilizador controlar o modelo dinâmico através do Raspberry Pi.

IK *Inverse Kinematics*; cinemática inversa, usada para converter coordenadas cartesianas em ângulos dos servos.

JSON *JavaScript Object Notation*; formato usado para guardar as configurações dos movimentos definidas na interface.

LiPo Bateria de polímero de lítio.

MG90S Servo motor de engrenagens metálicas com um grau de liberdade de aproximadamente 180°.

PCB *Printed Circuit Board*; placa de circuito impresso.

PEBA Filamento de impressão 3D de polieter-bloco-amida, elastómero termoplástico flexível e leve.

PETG *Polyethylene Terephthalate Glycol*; filamento de impressão 3D rígido, compatível com o processo FDM.

PLA Filamento de impressão 3D de ácido polilático, usado sobretudo pela facilidade de impressão e rigidez.

PVA *Polyvinyl Alcohol*; filamento de impressão 3D solúvel em água, considerado como material de suporte.

PWM *Pulse Width Modulation*; técnica usada para controlar os servos através da largura de pulso.

QR *Quick Response*; código usado para aceder rapidamente à interface móvel a partir de um telemóvel ou tablet.

RGB *Red, Green, Blue*; sistema de cor usado no LED de indicação de estado.

RTS *Request To Send*; sinal de controlo da porta série que pode provocar o reset do ESP32-S3.

SLA *Stereolithography*; tecnologia de impressão 3D por cura de resina fotopolimérica com luz ultravioleta.

SLS *Selective Laser Sintering*; tecnologia de impressão 3D por sinterização seletiva de material em pó.

STL Formato de ficheiro de malha 3D usado para representar a geometria de superfície da árvore brônquica antes da impressão.

TAC Tomografia Axial Computorizada; tecnologia de imagem médica usada para obter o modelo tridimensional da árvore brônquica.

TPC Filamento de impressão 3D de copoliéster termoplástico, material flexível usado em peças elásticas e funcionais.

TPE Filamento de impressão 3D de elastómero termoplástico, família de materiais com comportamento elástico semelhante a borracha.

TPU Filamento de impressão 3D de poliuretano termoplástico, usado em peças flexíveis e elásticas.

UART *Universal Asynchronous Receiver/Transmitter*; interface de comunicação série usada entre o Raspberry Pi e o ESP32-S3.

UPS *Uninterruptible Power Supply*; sistema de alimentação que mantém o equipamento ligado durante falhas ou interrupções temporárias da alimentação principal.

Lista de Figuras

3.1	Testes de impressão da árvore brônquica com TPU 95A, com suporte em PLA.	17
3.2	Escala de dureza Shore aplicada a filamentos flexíveis, com exemplos de objetos do dia a dia em pontos equivalentes da escala (fonte: Recreus). . .	18
3.3	Esquema consolidado de alimentação, comunicação e comando dos dois manipuladores delta.	26
3.4	Estrutura mecânica de um manipulador delta em PLA preto, antes da instalação dos servos, maio de 2026. A base circular e os três braços articulados estão já montados.	30
3.5	Os dois manipuladores delta montados com os servos Waveshare instalados, prontos para calibração de firmware, junho de 2026. Os cabos dos servos são visíveis mas ainda sem organização final de cablagem.	31
3.6	Caixa impressa em PLA branco que aloja o Raspberry Pi 4B e a fonte de alimentação comutada, junho de 2026.	31
4.1	Estrutura global das máquinas de estados e blocos de controlo.	33
4.2	Ambiente de desenvolvimento de software e eletrônica: portátil com PlatformIO, osciloscópio RIGOL, ESP32 em breadboard e tablet com diagramas de apoio, maio de 2026.	35
4.3	Trajectoria calculada para o movimento de batimento cardíaco, com representação 2D e 3D dos pontos de passagem.	38
4.4	Trajectoria calculada para o movimento respiratório, com representação 2D no plano ZX e visualização 3D.	39
4.5	Trajectoria calculada para a vibração associada à tosse, com representação 2D e 3D dos pontos de passagem.	41
4.6	Relação entre as estruturas de dados principais do firmware.	42
4.7	Esquema da FSM_Motion_Update.	45
4.8	Representação lateral das variáveis usadas na cinemática inversa do manipulador delta.	48
4.9	Relação linear de referência entre o ângulo calculado pela cinemática inversa e o pulso PWM aplicado aos servos nos dois manipuladores.	50

4.10	FSM.Serial_Read responsável pela leitura e validação dos dados recebidos por comunicação série.	53
4.11	FSM.Serial_Command responsável pela interpretação dos comandos recebidos por comunicação série.	54
5.1	Sessão de observação preliminar com profissional de saúde: broncoscópio introduzido no modelo de árvore brônquica impresso em 3D enquanto o sistema de controlo estava ativo, junho de 2026.	63

Lista de Tabelas

3.1	Componentes principais previstos para o sistema.	29
4.1	Movimentos considerados para a primeira versão do protótipo.	35
4.2	Parâmetros ajustáveis usados no cálculo dos movimentos na HMI.	40
4.3	Pontos calculados para os movimentos usados nos testes da interface. . . .	42
4.4	Comandos simples disponíveis na comunicação série.	51
4.5	Subcomandos do menu de leitura, teste e depuração.	52
4.6	Subcomandos de seleção do modo fisiológico.	52

Capítulo 1

Introdução

Este documento descreve o trabalho desenvolvido no âmbito de um projeto final de curso, dedicado à conceção de um modelo físico e dinâmico de árvore brônquica para treino de broncoscopia, apresentando de seguida o contexto que motivou esta escolha, os objetivos traçados e a forma como o presente relatório se encontra organizado.

1.1 Enquadramento

A broncoscopia flexível é um procedimento essencial em Pneumologia, Anestesiologia, Medicina Intensiva e Cirurgia Torácica, cuja execução exige coordenação psicomotora, reconhecimento anatómico, capacidade de navegação broncoscópica e tomada de decisão num ambiente clínico variável. Por esse motivo, a simulação tem sido proposta como uma etapa relevante no treino de novos médicos na área, permitindo suavizar a curva de aprendizagem, possibilitar treinos repetidos e aumentar a segurança antes da prática em doentes reais [1–5].

O projeto apresentado neste relatório teve origem numa proposta de trabalho elaborada pela equipa de Pneumologia do Centro Hospitalar Universitário do Algarve [6], que identificou a necessidade de um modelo de simulação de árvore brônquica, impresso em 3D e de baixo custo, para treino de broncoscopia flexível. A partir dessa proposta, o objetivo deste trabalho passou a ser o desenvolvimento de um modelo articulado de uma árvore brônquica, capaz de reproduzir, de forma legítima, os movimentos observáveis durante as broncoscopias, partindo da utilidade já reconhecida dos modelos físicos impressos em 3D e acrescentando-lhes uma componente dinâmica, de modo a aproximar a experiência de treino das condições reais de observação broncoscópica [7, 8].

O projeto foi dividido em duas secções complementares:

- O Modelo Dinâmico, que inclui todo o sistema de controlo e atuação motora, dividido em vários manipuladores delta, bem como o próprio modelo 3D da árvore brônquica;

- A HMI, ou interface homem-máquina, alojada num Raspberry Pi 4B com um monitor touchscreen, cuja função é permitir que o utilizador controle o modelo dinâmico de forma simples, intuitiva e natural, através de botões e menus dedicados, sem necessidade de conhecer os comandos da comunicação série entre os dois blocos.

1.2 Motivação

Existem vários simuladores comerciais de broncoscopia, contudo estes facilmente podem apresentar custos monetários extremamente elevados, o que limita a sua disponibilidade e acessibilidade em contextos de ensino e treino, razão pela qual a impressão 3D surge como uma forte alternativa, ao permitir criar modelos anatómicos acessíveis, personalizáveis e adaptáveis a diferentes níveis de dificuldade [7, 8]. Contudo, muitos destes modelos físicos permanecem estáticos, falhando na representação de fenómenos como a deslocação respiratória das vias aéreas, a variação de calibre dos brônquios, as pequenas oscilações induzidas pelo coração e as perturbações rápidas associadas à tosse, resultantes da resposta do corpo humano à inserção de um corpo externo nos brônquios [9–12].

A motivação central deste trabalho consistiu, portanto, na capacidade de combinar a acessibilidade e a versatilidade dos modelos físicos impressos em 3D com a possibilidade de implementação de movimentos anatómicos legítimos, comprovando assim a viabilidade de um simulador dinâmico e mecanicamente atuado, uma abordagem que, tanto quanto foi possível apurar junto da literatura disponível, ainda não se encontra documentada.

1.3 Objetivos

Este projeto pretendia investigar, desenvolver, construir e documentar um protótipo articulado de uma árvore brônquica para treino de videobroncofibroscopia.

Os seus objetivos específicos foram:

- estudar requisitos anatómicos, mecânicos e pedagógicos para os simuladores de broncoscopia;
- desenvolver um plano e uma estrutura física compatível com impressão 3D e atuação mecânica;
- integrar sensores ou atuadores e todo um sistema de eletrónica de controlo flexível, para os vários cenários de treino;
- implementar firmware e configurações pré-feitas de controlo para os diferentes perfis de movimento;
- desenvolver uma interface, HMI, para um controlo simples e intuitivo do modelo;

- definir uma estratégia de testes para avaliação da usabilidade, da repetibilidade e da robustez dos sistemas, bem como verificação do sincronismo dos movimentos em termos de tempo e de espaço percorrido pelos manipuladores robóticos;
- documentar as decisões de projeto, limitações e possíveis melhorias futuras.

1.4 Metodologia

Este projeto seguiu uma metodologia iterativa de desenvolvimento, começando por uma revisão bibliográfica alargada e pela definição dos requisitos anatómicos, mecânicos e pedagógicos do simulador. A primeira etapa de desenvolvimento centrou-se no modelo 3D da árvore brônquica, construído e refinado ao longo de várias iterações, validadas em conjunto com a equipa médica responsável pelo projeto. A partir daí, o restante hardware e o software foram desenvolvidos em paralelo e por camadas sucessivas, com protótipos intermédios a validar decisões de projeto antes de avançar para a fase seguinte, permitindo corrigir limitações identificadas em cada etapa em vez de as arrastar até ao final do processo. Esta abordagem culminou numa fase de testes e de validação experimental do protótipo final, descrita em detalhe no capítulo 5.

1.5 Organização do relatório

O relatório está organizado em seis capítulos:

- Capítulo 1: Apresenta o enquadramento, a motivação e os objetivos;
- Capítulo 2: Revisão do estado da arte, incluindo treino por simulação, impressão 3D e movimentos observáveis durante a broncoscopia;
- Capítulo 3: Descreve o desenvolvimento do hardware;
- Capítulo 4: Apresenta a arquitetura de software, os movimentos representados no protótipo, a cinemática inversa, as máquinas de estados, a comunicação série e todo o sistema por detrás da HMI;
- Capítulo 5: Reúne o plano de testes e identifica a informação experimental ainda em falta;
- Capítulo 6: Sintetiza as conclusões e melhorias futuras.

Capítulo 2

Estado da Arte

Antes de se justificar o desenvolvimento de um modelo físico e dinâmico de árvore brônquica, importa perceber onde se situa este trabalho relativamente ao que já existe em simulação médica para broncoscopia, pelo que este capítulo percorre o estado atual do conhecimento nesta área, desde o papel da simulação na aprendizagem da broncoscopia e o conceito de fidelidade que permite comparar diferentes simuladores entre si, até ao contributo da impressão 3D na criação de modelos anatómicos e aos principais movimentos fisiológicos observáveis durante o procedimento, que servem de referência direta ao trabalho desenvolvido nos capítulos seguintes.

2.1 Broncoscopia e treino por simulação

A broncoscopia flexível é um procedimento médico endoscópico que permite observar diretamente a árvore traqueobrônquica, recolher amostras, auxiliar diagnósticos e apoiar intervenções terapêuticas. A aprendizagem deste procedimento exige coordenação entre visão broncoscópica, manipulação fina do broncoscópio e interpretação anatómica. A simulação pormenorizada é, por isso, uma ferramenta relevante para repetir tarefas, treinar orientação espacial e reduzir a dependência inicial de treino em doentes reais [1–4].

As revisões e estudos de treino em broncoscopia apontam para ganhos na aquisição de competências quando os simuladores são integrados de forma estruturada no ensino. O valor pedagógico de um simulador depende não só do realismo visual, mas também da relação entre o exercício proposto, os objetivos de aprendizagem e a forma como o desempenho é avaliado [2, 3, 5].

Trabalhos mais recentes procuram comparar diretamente diferentes modalidades de treino entre si. Um estudo dinamarquês comparou quatro modelos distintos de treino em broncoscopia, incluindo modelos físicos e virtuais, concluindo que os modelos físicos de baixo custo se mantêm competitivos face a alternativas tecnologicamente mais sofisticadas e dispendiosas [13], enquanto outro estudo mostrou que sessões curtas de treino guiadas por um simulador produzem resultados de aprendizagem semelhantes aos obtidos sob

orientação direta de um tutor humano, reforçando o valor pedagógico da simulação mesmo sem supervisão constante [14].

2.2 Fidelidade dos simuladores

A fidelidade de um simulador pode ser entendida em várias dimensões:

- Fidelidade anatômica → refere-se à correspondência entre a geometria do modelo e a anatomia real, como à forma, proporção e organização espacial das vias aéreas;
- Fidelidade mecânica → refere-se à resistência ao avanço, flexibilidade e resposta ao contacto;
- Fidelidade funcional → refere-se à capacidade de reproduzir movimentos, variações de calibre e eventos transitórios como a tosse.

Esta distinção aproxima-se da lógica proposta por Miller para a avaliação de competências clínicas, que separa aquilo que um profissional sabe, sabe fazer, demonstra saber fazer e efetivamente faz na prática, situando os simuladores físicos precisamente na transição entre o demonstrar e o fazer [15]. Importa, no entanto, notar que a relação entre fidelidade e aprendizagem não é necessariamente linear, havendo trabalhos recentes que questionam se um maior realismo se traduz sempre em mais competência adquirida, sugerindo que o envolvimento ativo do formando pode pesar tanto quanto o rigor da réplica anatômica [16].

Na broncoscopia, um modelo estático pode ser útil para situações de treinos iniciais de navegação e reconhecimento anatômico. No entanto, durante um procedimento real, o operador encontra-se num ambiente vivo e dinâmico, em constante movimento. A respiração, os batimentos cardíacos, a tosse, a sedação e a própria interação do broncoscópio com as paredes internas exercem comportamentos e reações que modificam continuamente o ambiente e a sensação de controlo. Esta diferença cria oportunidade para os simuladores físicos de baixo custo que incorporem movimentos e fisiologia de forma controlada [1, 4, 8].

2.3 Simuladores de broncoscopia

Os atuais simuladores de broncoscopia recorrem a vários tipos de sistemas, nomeadamente treinos em ambientes puramente virtuais, em manequins comerciais, em modelos de bancada e, mais recentemente, em modelos impressos em 3D.

Os sistemas virtuais, como o simulador ORSIM, permitem gerar cenários de dificuldade variável e obter métricas automáticas de desempenho, tendo demonstrado boa validade e fiabilidade na avaliação de competências em broncoscopia flexível, distinguindo

de forma consistente entre principiantes e peritos [17]. Contudo, estes sistemas dependem de hardware háptico dedicado para simular a resistência ao avanço e o contacto com as paredes das vias aéreas, um aspeto que continua a ser tecnicamente difícil de reproduzir com fidelidade, uma vez que a deteção de força na interação entre instrumento e tecido em endoscopia minimamente invasiva (termo mais abrangente, que inclui a broncoscopia como um dos seus procedimentos) permanece uma área ativa de investigação, sem uma solução consensual e amplamente adotada [18]. Modelos físicos baseados em tecido biológico real, como um biosimulador reutilizável construído a partir de pulmões suínos ventilados, procuram contornar esta limitação ao oferecer uma resposta tátil genuína, tendo sido avaliados favoravelmente por broncoscopistas experientes e principiantes em comparação direta com um simulador virtual comercial [19]. Estes modelos biológicos, no entanto, não são reutilizáveis indefinidamente, exigem preservação especializada e apresentam variabilidade anatômica de espécime para espécime, o que limita a sua padronização enquanto ferramenta de ensino e de avaliação objetiva.

Esta lacuna, entre a normalização própria dos sistemas virtuais e o realismo tátil dos modelos biológicos, abre espaço para os modelos impressos em 3D, que procuram oferecer geometria anatômica consistente e reproduzível, associada a materiais capazes de aproximar a resposta mecânica dos tecidos reais.

2.4 Impressão 3D aplicada à simulação médica

A impressão 3D é uma tecnologia de produção de objetos tridimensionais, realizada através da adição de material camada a camada, podendo o método de adição ou aglomeração dos materiais variar bastante consoante a tecnologia utilizada, como FDM, SLS ou SLA. A comparação técnica entre estes processos é retomada no desenvolvimento do modelo físico, onde a escolha da tecnologia de fabrico passa a ter impacto direto na geometria, no material e na viabilidade de impressão da árvore brônquica. Com o passar dos anos e o amadurecimento da tecnologia e dos materiais disponíveis, este método de produção veio proporcionar uma oportunidade inovadora na criação de estruturas e geometrias complexas, a um baixo custo e numa fração do tempo face aos outros métodos de produção, afetando todas as áreas científicas e industriais, incluindo a medicina. Esta permite a produção de protótipos anatômicos com custo relativamente baixo, a correção e o ajuste de dimensões em pouco tempo e o teste de materiais inovadores que, com outros métodos, seria impensável, tanto pelo custo de produção como pela complexidade dos processos.

A utilização clínica e educativa da impressão 3D em modelos anatômicos exige, contudo, um controlo de qualidade rigoroso ao longo de todo o processo, desde a segmentação da imagem médica até à impressão final, sendo possível identificar e quantificar erros próprios de cada etapa, como o erro de segmentação, o erro de edição digital e o

erro de impressão propriamente dito, cuja soma determina a fidelidade geométrica final do modelo obtido [20]. Esta preocupação com o rigor dimensional não se limita a modelos de treino, estendendo-se também a aplicações clínicas diretas, como o desenho e o fabrico de stents personalizados para o tratamento de estenoses das vias aéreas, benignas ou malignas, a partir da anatomia específica de cada doente [21–23], ou ainda à produção de modelos de traqueia para treino de cricotiroidotomia de emergência e de modelos de vias aéreas superiores para treino de entubação endotraqueal, ambos criados com o objetivo de tornar este tipo de treino mais acessível e menos dependente de equipamento comercial dispendioso [24, 25].

2.4.1 Aplicação em simuladores de árvores brônquicas

No caso dos simuladores de árvores brônquicas, os modelos impressos em 3D ganharam uma grande importância por permitirem transformar dados anatômicos, obtidos por exemplo a partir de exames de tomografia computadorizada (TAC), em geometrias físicas. Em broncoscopia, esta abordagem pode representar a árvore traqueobrônquica com boa correspondência espacial e permitir a criação de simuladores específicos para treino de navegação [7, 8], ainda que a sua principal limitação, quando usados de forma isolada, seja a ausência de movimento e de resposta fisiológica. Modelos multi-material, combinando simultaneamente componentes rígidos e flexíveis numa só impressão, têm vindo a ser explorados precisamente para aproximar a experiência tátil da broncoscopia real sem comprometer a robustez estrutural do conjunto [26], enquanto outras abordagens recorrem à impressão a cores, atribuindo uma cor distinta a cada segmento ou ramo brônquico, de modo a tornar mais perceptível a organização da árvore traqueobrônquica e a facilitar simultaneamente o treino de navegação e a aprendizagem da nomenclatura anatômica [27]. Contudo, a ideia de impressão destes modelos apresenta desafios e detalhes muito importantes, tais como a escolha do material, pois facilmente influencia o grau de dificuldade de impressão, a flexibilidade, a resistência, a sensação ao toque e a disponibilidade de cor. Materiais rígidos podem servir para validar forma e montagem, enquanto materiais flexíveis podem aproximar-se melhor à resposta mecânica, ainda que aumentem consideravelmente a dificuldade de impressão e calibração. Assim, a impressão 3D deve ser entendida também como uma plataforma de iteração, não apenas como uma técnica de fabrico final.

2.5 Movimentos observáveis durante a broncoscopia

A árvore traqueobrônquica não se comporta como uma estrutura fixa durante uma broncoscopia. Mesmo quando o broncoscópio se mantém praticamente parado, a imagem observada pode apresentar deslocações lentas, alterações de orientação, variações do lúmen

e perturbações rápidas. Para o operador, estes movimentos não aparecem isolados:

- A respiração cria uma componente periódica dominante;
- O batimento cardíaco acrescenta pequenas oscilações locais;
- A tosse introduz eventos abruptos;
- O contacto do broncoscópio com a parede pode modificar temporariamente a posição ou a forma do segmento observado.

Do ponto de vista de um simulador físico, estes fenómenos não precisam de ser reproduzidos com toda a complexidade fisiológica para terem valor pedagógico, mesmo que acabem sempre por apresentar um maior realismo. O mais importante é que o utilizador deixe de observar um modelo completamente estático e passe a treinar num ambiente com movimento por vezes imprevisível e suficientemente coerente com o que é visto durante o procedimento real. Por isso, os movimentos descritos de seguida foram analisados quanto à sua relevância visual, dificuldade de implementação e utilidade, seguindo a categorização e os intervalos de frequência e amplitude propostos num documento de trabalho elaborado especificamente para este projeto pela equipa de Pneumologia do Centro Hospitalar Universitário do Algarve [28], que serviu de base clínica direta às secções seguintes.

2.5.1 Movimento respiratório

O movimento respiratório é a componente mais frequente e previsível, ocorrendo durante o processo cíclico da inspiração e expiração, descrito como a deslocação dos pulmões com predominância craniocaudal, acompanhado por pequenas componentes laterais e por alterações locais da geometria das vias aéreas. A frequência respiratória normal situa-se aproximadamente entre 12 e 20 ciclos por minuto, embora possa variar com ansiedade, sedação, esforço respiratório, patologia e contexto clínico.

Análises mostram que a fase respiratória pode alterar a posição de estruturas pulmonares e influenciar modelos de fluxo nas vias aéreas [11, 12], existindo ainda, além da deslocação global, variações do calibre brônquico ao longo do ciclo respiratório, um fenómeno já descrito por trabalhos clássicos que demonstraram os mecanismos das alterações de calibre dos brônquios durante a respiração normal e das suas variações rítmicas [9, 10]. Mais recentemente, técnicas de modelação computacional têm procurado prever e simular este movimento de forma específica ao paciente, recorrendo a modelos de elementos finitos capazes de estimar o deslocamento dos brônquios ao longo do ciclo respiratório a partir de imagem médica, alcançando erros de previsão inferiores a 3

2.5.2 Movimento cardíaco

O movimento cardíaco introduz oscilações mais rápidas e de menor amplitude, com intervalos regulares entre ocorrências, cuja influência tende a ser mais perceptível em regiões próximas do coração, em particular junto ao brônquio principal esquerdo e a estruturas adjacentes. Ao contrário da respiração, que domina a posição global da árvore brônquica, o batimento cardíaco surge antes como uma vibração ou perturbação sobreposta.

Estudos com imagem 4D confirmam esta distinção, tendo quantificado separadamente as contribuições respiratória e cardíaca no movimento de estruturas pulmonares e observado uma influência cardíaca proporcionalmente maior em lesões centrais e superiores, mais próximas do coração e dos grandes vasos [29]. Esta influência não se limita à posição da árvore brônquica, estendendo-se também ao próprio fluxo de ar no seu interior. Imagem dinâmica obtida por radiação de sincrotrão revelou que o batimento cardíaco gera oscilações de pressão e de fluxo de ar mensuráveis nos brônquios principais, num fenómeno localizado junto à carina até então difícil de demonstrar experimentalmente [30]. Em casos anatómicos particulares, esta proximidade pode mesmo traduzir-se em compressão mecânica direta do brônquio principal esquerdo por estruturas cardiovasculares adjacentes, como a aorta descendente ou uma artéria pulmonar dilatada, um efeito documentado em população pediátrica com anomalias intracardíacas [31].

Num modelo de treino, esta componente é útil para quebrar a sensação de movimento puramente sinusoidal e para acrescentar uma perturbação de menor amplitude, pelo que, no contexto deste projeto, o batimento cardíaco não foi tratado como uma reconstrução anatómica exata da interação coração-pulmão, mas antes como um perfil adicional de movimento, mais rápido e discreto, combinado com a respiração.

2.5.3 Tosse e perturbações rápidas

A tosse é um evento curto, abrupto e menos previsível, provocando, durante uma broncoscopia, a deslocação súbita das vias aéreas, com tendência para a ocorrência de réplicas adjacentes à deslocação principal, alteração temporária do campo visual, variação do fluxo de ar e perda momentânea de orientação. Para quem treina, este tipo de evento é importante porque obriga a recuperar a posição visual e a manter controlo do broncoscópico após uma perturbação.

A tosse é, em si mesma, um reflexo vital complexo, composto por uma fase inspiratória, uma fase compressiva com encerramento da glote e uma fase expiratória explosiva, coordenada por recetores das vias aéreas e por vias nervosas aferentes e eferentes bem descritas na literatura fisiológica [32]. A modelação computacional recente deste fenómeno tem-se debruçado sobre a dinâmica de gotículas e da película mucosa nas vias aéreas superiores durante um evento de tosse, com o objetivo de relacionar as suas características com diferentes patologias pulmonares [33], o que confirma o interesse científico atual em

compreender este movimento para além da sua simples deteção visual durante um procedimento.

No simulador, a tosse deve por isso ser representada como um evento transitório, com duração reduzida e amplitude superior à do batimento cardíaco, podendo a sua ocorrência ser programada de forma aleatória dentro de limites definidos, o que permite criar cenários de treino mais realistas sem obrigar à reprodução completa da fisiologia da tosse.

2.5.4 Interação com o broncoscópio

A passagem do broncoscópio também influencia a geometria observada, dado que o contacto com as paredes pode causar deformações locais, resistência ao avanço, pequenas oscilações e alterações da orientação da imagem. Esta componente é especialmente importante para o realismo tátil, porque aproxima o simulador da sensação física de navegação no interior das vias aéreas.

Apesar da sua relevância, esta interação mostra-se a mais difícil de implementar, exigindo materiais flexíveis adequados, geometrias calibradas com espessura e resistência adequadas e, idealmente, sensores ou mecanismos capazes de responder ao contacto. Esta dificuldade é também reconhecida na investigação em robótica endoscópica (área mais ampla, da qual a robótica aplicada à broncoscopia faz parte), onde a deteção de força na interação entre instrumento e tecido continua a ser identificada como um dos principais desafios em cirurgia minimamente invasiva [18], tendo já sido exploradas soluções como broncoscópios robóticos de balão, especificamente concebidos para procedimentos nas vias aéreas e capazes de aplicar forças controladas sobre as suas paredes sem lesionar o tecido, ainda que este tipo de solução acrescente complexidade mecânica significativa ao sistema [34]. Esta complexidade torna particularmente difícil a implementação deste tipo de fenómeno num simulador.

2.5.5 Patologias e deformações locais

Estenoses, obstruções, variações anatómicas e alterações patológicas modificam a navegação e o aspeto interno das vias aéreas, sendo que a impressão 3D permite criar geometrias específicas para representar alguns destes cenários de forma estática, o que já pode ser útil para treino de reconhecimento anatómico e planeamento de procedimentos. Um exemplo clínico bem documentado deste tipo de fenómeno é a traqueobroncomalácia, uma condição em que o amolecimento da cartilagem traqueal e brônquica provoca colapso dinâmico excessivo das vias aéreas, observado por videobroncoscopia em cerca de 69

No entanto, a simulação ativa de estenose variável, colapso local ou deformação dinâmica do lúmen exige um conjunto complexo de atuadores distribuídos, materiais compatíveis e uma estratégia de controlo mais complexa.

2.6 Síntese crítica

Esta análise comprova que existem necessidades que nem sempre são satisfeitas em simultâneo. Por um lado, os simuladores devem ser suficientemente acessíveis para poderem ser usados em contexto académico e profissional, permitindo treinos repetidos. Por outro, devem evitar uma representação demasiado estática das vias aéreas, porque a broncoscopia real ocorre num ambiente anatómico em constante movimento.

Os modelos impressos em 3D respondem bem à primeira necessidade, pois permitem criar geometrias anatómicas de baixo custo e iterar rapidamente o projeto, como demonstram diversos exemplos comerciais e académicos de árvores brônquicas estáticas, alguns já disponíveis há várias décadas [7, 26, 27, 35–37]. Contudo, quando usados isoladamente, tendem a limitar-se à fidelidade geométrica.

Algumas publicações têm procurado atenuar exatamente esta limitação, conferindo algum grau de dinamismo a modelos impressos em 3D e a outras réplicas físicas de vias aéreas, embora nenhuma delas recorra a atuação mecânica da própria estrutura anatómica. Um primeiro exemplo consiste na insuflação controlada de ar ou soro através de uma porta lateral, fazendo com que o próprio modelo impresso se expanda e contraia de forma passiva, uma solução já aplicada a um protótipo de via aérea impresso em 3D capaz de emular movimento fisiológico e patológico [8]. Outro trabalho, mais próximo da atuação robótica, recorre a seringas acionadas por uma guia linear motorizada para empurrar ar através de condutas rígidas que representam a árvore brônquica e a traqueia, replicando fisicamente tanto a respiração como a tosse. Contudo, mesmo nesta solução, os brônquios e a traqueia permanecem estruturas rígidas e estáticas, sendo apenas o fluxo de ar no seu interior que é mecanicamente controlado, sem qualquer tentativa de reproduzir o batimento cardíaco [38].

Fora do domínio da broncoscopia, outros trabalhos recorrem a atuação mecânica real para simular a mecânica ventilatória, mas fazem-no ao nível do diafragma ou da parede torácica, não da própria árvore brônquica. É o caso de um simulador respiratório que combina músculos artificiais pneumáticos com um diafragma sintético para insuflar pulmões reais ou artificiais [39], de um simulador eletromecânico de pulmão que combina componentes poliméricos e tecido pulmonar orgânico para reproduzir uma respiração realista [40] e de modelos deformáveis usados em radioterapia para replicar mecanicamente a expansão e a contração de um pulmão, sem qualquer representação da árvore brônquica no seu interior [41]. À mesma categoria pertencem ainda simuladores pulmonares controlados por motor de passo ou por pistão servo-controlado, desenvolvidos sobretudo para testar e calibrar ventiladores mecânicos [42, 43], bem como patentes que atuam mecanicamente a parede torácica de manequins de treino, por bolsas de ar ou por sistemas biela-manivela, para simular respiração ou manobras de reanimação [44, 45]. Outros dispositivos comerciais patenteados representam a árvore brônquica de forma mais completa, incluindo

canais para brônquio esquerdo, direito e traqueia, mas destinam-se a treinar procedimentos de drenagem postural com sensores de posição, não a reproduzir movimento fisiológico [46]. Uma última categoria de dispositivos recorre efetivamente a atuadores mecânicos reais sobre estruturas anatómicas de treino, mas aplicados à via aérea superior, como a mandíbula, a língua ou a articulação temporomandibular em simuladores de gestão de via aérea e de entubação orotraqueal, nunca chegando a atuar a árvore brônquica em profundidade e as suas bifurcações [47].

Do lado puramente computacional, têm também sido propostos métodos capazes de simular visualmente a deformação respiratória das vias aéreas durante a navegação broncoscópica, por renderização de imagem em tempo real a partir de dados de tomografia computadorizada, as imagens médicas que descrevem em cortes transversais a anatomia interna do paciente, mas sem qualquer correspondência física. O resultado é sempre uma simulação gráfica, não um modelo tangível que o formando possa manipular com um broncoscópio real [48]. Já no campo da robótica cirúrgica aplicada à broncoscopia, os poucos manipuladores desenvolvidos destinam-se a operar sobre vias aéreas reais de doentes ou de cadáveres, como ferramentas de tratamento e não como modelos de treino, permanecendo a anatomia sempre estática durante a sua atuação [49]. Mesmo alargando esta análise a protótipos ibero-americanos, como um simulador de broncoscopia com realimentação háptica desenvolvido no México, cuja árvore brônquica nunca chega a ser impressa fisicamente e permanece apenas como um modelo gráfico renderizado [50], a conclusão mantém-se a mesma.

Em síntese, não se encontrou em toda a literatura revista nenhum modelo de árvore brônquica impresso em 3D em que a própria estrutura anatómica seja mecanicamente atuada, por manipuladores robóticos ou mecanismo equivalente, de forma a reproduzir de forma simultânea os principais fenómenos fisiológicos observáveis durante a broncoscopia. É precisamente esta lacuna que o presente projeto procura preencher.

A componente dinâmica, mesmo que simplificada, acrescenta valor ao treino, pois obriga o utilizador a lidar com deslocações por vezes imprevisíveis e com a perda parcial do enquadramento, devido a perturbações temporárias.

Assim, o objetivo deste projeto encontra-se em combinar uma árvore brônquica física, obtida por impressão 3D, com uma arquitetura de atuação capaz de reproduzir de forma controlada os fenómenos anatómicos dinâmicos identificados ao longo deste capítulo, respondendo a uma lacuna que a literatura revista ainda não preenche.

Capítulo 3

Desenvolvimento do Hardware

Este capítulo descreve o percurso de desenvolvimento do hardware, desde as primeiras impressões de teste até à montagem final do sistema dos dois manipuladores delta, eletrónica de controlo e suportes para guardar os componentes de forma segura. O processo não foi linear. Cada iteração revelou limitações que levaram a novas abordagens do design, materiais e arquitetura mecânica. O que se segue documenta esse caminho de escolhas intermédias, bem os problemas que as motivaram.

3.1 Requisitos do sistema

Como já referido o modelo deve permitir o treino de videobroncofibroscopia com um modelo físico de baixo custo, robusto, (lavável) e fácil de montar, tendo os requisitos principais sido agrupados em quatro eixos:

- → Realismo anatómico;
- → Facilidade de utilização;
- → Movimento fisiológico controlado;

No plano anatómico, o modelo deve representar a árvore brônquica com dimensões aproximadas, ângulos de ramificação e características compatíveis com dados obtidos por tomografia computadorizada, enquanto no plano mecânico deve permitir a introdução de movimentos de respiração, batimento cardíaco e tosse sem comprometer a estabilidade do conjunto. No plano pedagógico, deve ainda permitir a repetição de exercícios e a adaptação do grau de dificuldade e, no plano técnico, (manter a solução acessível, evitando materiais, atuadores ou processos de fabrico que tornem o sistema demasiado caro para o objetivo do projeto).

Do ponto de vista funcional, o sistema foi separado em dois blocos:

- 1º → Modelo Dinâmico, constituído pela árvore brônquica impressa em 3D, pelos manipuladores delta, pelos servo-motores e seus respetivos drivers e pelo mi-

crocontrolador ESP32-S3, executando fisicamente os movimentos e mantendo um comportamento previsível mesmo quando recebe comandos externos;

- 2º → HMI, suportada pelo Raspberry Pi 4B e pelo monitor touchscreen, apresentando ao utilizador, através de botões e menus intuitivos, comandos de alto nível, como iniciar, parar, escolher modo fisiológico ou ajustar parâmetros, ficando a cargo do software a tradução desses comandos, o cálculo e a geração de coordenadas para os movimentos pretendidos e a execução física dos mesmos, o que torna o sistema mais simples e intuitivo para o utilizador;

Um dos requisitos mais importantes do projeto é a capacidade de reproduzir movimentos fisiológicos de forma controlada, com frequências e amplitudes compatíveis com os valores observados em humanos, tornando-se também o desafio mais importante deste projeto, pois ambos os sistemas têm que funcionar de forma paralela e sincronizada, sem comprometer a precisão temporal dos movimentos sobrepostos nem o sincronismo entre os comandos enviados pela interface e a resposta física do modelo dinâmico.

3.2 Modelo físico da árvore brônquica

3.2.1 Técnicas de impressão 3D

Entre as tecnologias de impressão 3D mais utilizadas destacam-se o FDM, a SLA e a SLS, cada uma assente num princípio de fabrico distinto e, por isso, mais adequada a diferentes tipos de aplicação.

O FDM (Fused Deposition Modeling, ou modelação por deposição de material fundido) é a tecnologia mais acessível e mais difundida, consistindo na extrusão de um filamento termoplástico aquecido através de um bico, que deposita o material camada a camada sobre uma base de impressão. É compatível com uma grande variedade de materiais, como o PLA, o PETG, o ABS ou filamentos flexíveis como o TPU, permitindo a produção de peças estruturalmente resistentes a um custo reduzido, ainda que com uma resolução e um acabamento de superfície mais limitados, sendo geralmente visíveis as linhas de cada camada. Para geometrias com zonas em consola ou sem apoio inferior, o FDM necessita ainda de estruturas de suporte, que têm de ser removidas após a impressão.

A SLA (Stereolithography) recorre a uma cuba de resina fotopolimérica líquida, curada seletivamente camada a camada por uma fonte de luz ultravioleta, um laser ou um projetor. Esta tecnologia permite obter peças com elevada resolução e um acabamento de superfície muito mais fino do que o FDM, sendo por isso indicada para modelos com geometrias detalhadas. Em contrapartida, exige equipamento de pós-processamento dedicado, nomeadamente para lavagem das peças em álcool isopropílico e cura adicional sob luz ultravioleta, além de recorrer a materiais mais dispendiosos e, em geral, mais frágeis

do que os filamentos termoplásticos.

Por fim, a SLS (Selective Laser Sintering) utiliza um laser para sinterizar seletivamente um material em pó, tipicamente uma poliamida, camada a camada dentro de um leito de pó. Uma das suas principais vantagens é dispensar estruturas de suporte dedicadas, uma vez que o próprio pó não sinterizado sustenta as zonas em consola durante a impressão, permitindo fabricar geometrias mais complexas. As peças resultantes apresentam ainda propriedades mecânicas mais uniformes do que as obtidas por FDM. Contudo, o equipamento e os materiais são substancialmente mais caros, o que torna esta tecnologia mais comum em contexto industrial do que em pequenos projetos ou protótipos.

No contexto deste projeto, esta comparação foi particularmente relevante na escolha entre FDM e SLA para a impressão da árvore brônquica, decisão discutida em detalhe na section 3.2.5.

3.2.2 O modelo STL da árvore brônquica

O ponto de partida do projeto foi o modelo tridimensional da árvore brônquica, obtido a partir de dados de uma tomografia computadorizada (TAC), a tecnologia de imagem médica utilizada pela equipa clínica para obter o scan 3D, que foi realizada e entregue pelo grupo de médicos encarregues do projeto, em formato de malha 3D (STL). Este tipo de ficheiro define apenas a geometria da superfície do objeto, sob a forma de uma malha de triângulos, sem qualquer volume ou espessura associada, pelo que o modelo obtido correspondia apenas à superfície interna da árvore brônquica, uma espécie de casca oca, sendo por isso necessário atribuir-lhe espessura manualmente antes da impressão 3D.

3.2.3 Programas utilizados no tratamento do modelo

Esta operação foi feita inicialmente no programa gratuito Tinkercad (da Autodesk), com o objetivo de realizar um teste rápido em PLA e confirmar a viabilidade geométrica da impressão. Depois de uma confirmação visual, passou-se para um programa mais indicado para este trabalho, o Fusion 360, um programa CAD da Autodesk destinado ao desenho 3D de sólidos, superfícies e malhas, que dispõe de uma ferramenta interna chamada “Forms”. Ao contrário do ambiente normal do Fusion 360, onde se trabalha maioritariamente com objetos sólidos e geometria bem definida, o Forms é exclusivamente indicado para trabalhar diretamente com scans 3D e as suas irregularidades geométricas, o que o tornava particularmente adequado a este caso.

No ambiente Forms, não se edita diretamente o ficheiro de malha. Em vez disso, é criada uma aproximação à superfície do ficheiro através de uma camada virtual, sendo apenas a essa camada que depois é possível atribuir um valor de espessura, criando assim um verdadeiro objeto 3D da árvore brônquica. Esta abordagem permitia editar em tempo real a geometria da camada e corrigir irregularidades existentes na malha, bem

como descartar secções consideradas desnecessárias, como por exemplo as extremidades mais pequenas do scan 3D, que não tinham qualquer utilidade para o modelo devido à incompatibilidade entre a largura das vias nessas extremidades e a largura da sonda broncoscópica.

Contudo, observou-se rapidamente que este método era demorado e ineficiente. O ambiente Forms trabalha com formas primitivas do tipo T-Spline, como o cilindro sem base nem topo ou uma superfície do tipo folha, compostas por vértices, arestas e faces que podem ser deslocados, rodados ou escalados através da ferramenta Edit Form, moldando a forma até se aproximar da geometria pretendida. Para este modelo, o cilindro sem tampas foi a forma escolhida, por já se aproximar naturalmente do formato dos ramos da árvore brônquica. Não era, no entanto, possível aplicar esta aproximação ao modelo completo de uma só vez. Era necessário trabalhar ramo a ramo, ajustando individualmente um destes cilindros a cada ramificação através da ferramenta Edit Form, para depois ser cosido aos ramos adjacentes e ter a posição dos pontos novamente afinada, de forma a manter a fidelidade à geometria original do scan. Este processo, repetido ramo a ramo ao longo de toda a árvore brônquica e seguido de sucessivas costuras e correções, tornava o método muito demorado. Por esta razão, optou-se por um outro programa, o MeshMixer, também da Autodesk, direcionado exclusivamente para trabalhar e modificar diretamente ficheiros de malha 3D. Ao contrário do Forms, que edita apenas uma camada virtual de aproximação, o MeshMixer permite selecionar diretamente os triângulos da malha através de ferramentas de pincel e laço, corrigindo na fonte as imperfeições e cortando as secções desnecessárias. A ferramenta Inspector fecha automaticamente as malhas abertas e repara falhas de continuidade na superfície, enquanto o Offset permite atribuir espessura diretamente à malha, sem depender de uma camada intermédia como no Forms. Uma outra vantagem do MeshMixer em relação ao Forms está na forma como lida com sobreposições. Quando, com o aumento de espessura (scale-up), duas malhas ficam sobrepostas, a ferramenta Boolean Union do MeshMixer une-as automaticamente numa só peça, enquanto o Fusion 360 não consegue processar essas sobreposições e obriga a manter uma distância mínima entre elas, exigindo várias iterações e ajustes manuais.

Mesmo depois da transição para o MeshMixer, o Fusion 360 continuou a ser bastante utilizado ao longo da modelação da árvore brônquica, uma vez que continuava a ser necessário desenhar as peças de encaixe que permitiam ligar o modelo à estrutura de suporte dos manipuladores. O fluxo de trabalho consistia em tratar o modelo no MeshMixer, atribuindo-lhe espessura e cortando os limites dos tubos necessários, para depois o exportar em formato STL e importá-lo no Fusion 360, onde não era feita qualquer conversão ou tratamento adicional da malha. Nesse ambiente, desenhavam-se apenas as peças de encaixe como objetos sólidos independentes, posicionados de forma a manterem-se dentro dos limites internos da malha, sem nunca ultrapassar para o lado oco do modelo.

O ficheiro completo, incluindo a malha importada e as peças de encaixe recém-

desenhadas, era depois exportado em conjunto como um único STL, de forma a evitar que o programa mantivesse os dois tipos de objeto como entidades distintas, o que poderia criar problemas mais tarde na importação para o slicer. Esta união era possível devido à própria natureza do formato STL, que, ao contrário de formatos como o STEP ou o OBJ, não transporta qualquer informação sobre a estrutura interna ou a separação entre objetos de um ficheiro, sendo composto apenas por coordenadas de vértices e pela sua conectividade em triângulos. Um ficheiro STL binário, em particular, nem sequer tem capacidade para representar mais do que um único conjunto de triângulos, pelo que exportar a malha e as peças sólidas em conjunto resultava sempre numa única superfície contínua, sem qualquer separação reconhecível entre os dois tipos de objeto originais.

3.2.4 Seleção de materiais



Figura 3.1: Testes de impressão da árvore brônquica com TPU 95A, com suporte em PLA.

A seleção de materiais foi considerada uma decisão crítica ao longo de todo o projeto, tendo sido identificados filamentos flexíveis como PEBA, TPU, TPE e TPC, filamentos especiais de dureza variável utilizados mais tarde no projeto e diferentes durezas comerciais, por exemplo 95A, 85A e 70A. Como o objetivo do modelo era aproximar-se o mais possível da realidade anatómica, era necessário utilizar um material suficientemente mole, (sem que se soubesse à partida qual o grau de maciez adequado), pelo que foi necessário testar vários materiais, geometrias e durezas ao longo do projeto.

Nos filamentos maleáveis, quanto menor a dureza do material, mais difícil se torna a sua impressão. Os filamentos mais duros alimentam-se com facilidade através do extrusor, enquanto os mais moles tendem a comprimir-se e a deformar-se nas engrenagens de alimentação em vez de serem corretamente empurrados, o que aumenta significativamente a tendência para entupimentos e para defeitos de impressão, como o stringing. O TPU exige ainda um controlo mais apertado de vários parâmetros de impressão, nomeadamente

e ramificadas, um dos problemas que mais viria a afetar a impressão em FDM, como descrito mais adiante nesta secção. Permite também obter um acabamento de superfície muito mais fino e sem linhas de camada visíveis, característica relevante para reproduzir com fidelidade a superfície interna lisa da árvore brônquica. Além disso, existe atualmente uma grande variedade de resinas flexíveis e de silicones fotocuráveis compatíveis com esta tecnologia, com durezas Shore A que vão de aproximadamente 80A [51] até valores bastante mais baixos, como 70A [52], 63A ou 55A [53], 50A [54] ou mesmo 40A em resinas de silicone puro [55], um intervalo mais alargado e mais fácil de alcançar de forma fiável do que o observado nos filamentos de FDM, em que durezas abaixo de 70A se tornam cada vez mais problemáticas de imprimir, precisamente porque o processo de cura de uma resina líquida não depende da extrusão mecânica de um filamento sólido através de engrenagens e de um bico;

- Desvantagens do SLA: equipamento com custo muito superior e dependente de equipamento adicional de pós-processamento, nomeadamente lavagem das peças em álcool isopropílico e cura adicional sob luz ultravioleta, o que tornava a sua aquisição incomportável para o projeto. Os volumes de impressão são também tipicamente mais reduzidos do que os disponíveis em impressoras FDM, o que poderia condicionar a impressão do modelo completo da árvore brônquica numa só peça.

Mantendo-se a impressão em FDM, foram equacionadas três opções para a disposição do modelo:

1. **Impressão em peça única com suportes exteriores.** O modelo seria impresso de uma só vez, com suportes colocados do lado de fora para sustentar as zonas sem apoio inferior, que ficariam essencialmente a ser impressas “no ar”:
 - Vantagens: simplicidade de modificação da impressão e possibilidade de reaproveitar os próprios suportes como estrutura de fixação do modelo à base com os atuadores;
 - Desvantagens: tempo de impressão elevado, uma vez que todo o modelo seria impresso de uma só vez, obrigando a reimprimir a peça inteira em caso de falha, com desperdício considerável de filamento, tempo e energia, além de os suportes só poderem ser colocados no exterior do modelo, deixando sem sustentação zonas interiores igualmente sujeitas a grandes deformações, sobretudo em materiais mais moles.
2. **Desdobramento e impressão individual das ramificações.** Esta opção passava por separar literalmente cada ramificação da malha num objeto independente

e desdobrar a secção cilíndrica de cada ramo num plano, à semelhança do desdobramento de papel para construir um sólido, atribuindo-lhe espessura já na forma planificada e devolvendo-lhe a forma tridimensional apenas depois de impresso:

- Vantagens: possibilidade de modificar facilmente cada ramo de forma individual e de integrar sensores ou atuadores diretamente no modelo, caso necessário;
- Desvantagens: incerteza quanto à viabilidade do método, uma vez que, embora fosse possível desdobrar um ficheiro STL num plano com programas como o Blender, não havia garantia de que o mesmo funcionasse com um scan tão complexo e detalhado, podendo gerar erros ou geometrias inesperadas. A montagem após a impressão era também um risco, dado que os filamentos maleáveis são imprevisíveis e podem esticar ao serem retirados da base de impressão, criando folgas na união das peças e exigindo uma estratégia própria de costura entre secções, com uma probabilidade reduzida de sucesso ao longo de tantas ramificações.

3. **Moldação por vazamento em molde negativo.** Em vez de imprimir diretamente o modelo, esta opção previa a impressão de um molde negativo, utilizado depois para vazar resina ou silicone e obter o positivo da árvore brônquica. Foi uma ideia forte durante grande parte do projeto, funcionando quase sempre como plano secundário:

- Vantagens: possibilidade de usar um material mais rígido no molde e um material extremamente mole no modelo positivo, bem como reutilizar o mesmo molde para produzir vários modelos sem o descartar;
- Desvantagens: como a árvore brônquica é oca, era necessário produzir um molde negativo tanto para a superfície exterior como para o espaço interior do modelo, ficando o positivo reduzido a uma película fina, com a espessura pretendida, encaixada entre esses dois moldes negativos. Isto obrigava a unir com precisão as três secções de molde em volta dessa película, o negativo interior e os dois lados do negativo exterior. A orientação das ramificações em várias direções dificultava ainda a remoção dos moldes sem danificar o modelo. Por fim, a geometria complexa e extensa não dava garantias de que fosse possível injetar resina ou silicone de forma uniforme em todos os cantos, com risco de inconsistências na espessura das paredes.

Mais tarde, motivadas pela futura aquisição de uma impressora Bambu Lab com dois bicos de impressão independentes (assunto detalhado mais adiante), foram ainda teorizadas duas variações aos métodos já descritos:

1. **Suportes solúveis em PVA**, uma variação da impressão em peça única com suportes exteriores, substituindo o material dos suportes pelo filamento solúvel PVA:

- Vantagens: por ser solúvel em água, permitiria colocar suportes também no interior do modelo, em regiões críticas, aumentando a qualidade de impressão sem complicar o processo de remoção;
- Desvantagens: o PVA tem um custo por quilo superior ao do PLA e mesmo ao do TPU, já de si caro nas durezas mais baixas. Por ser sempre perdido após a dissolução em água, o custo de utilização tornar-se-ia incomportável. Esta variação acabou por nunca ser testada, por falta de tempo para adquirir o material a tempo.

2. **Molde negativo interno com vazamento por imersão**, uma variação do método dos moldes, recorrendo a uma resina especial de uso médico que solidifica em contacto com o ar, mantendo uma dureza específica de acordo com o tempo de exposição antes de a reação ser interrompida com um químico próprio. O processo consistiria em imprimir apenas o molde negativo interno da árvore brônquica, num material de fácil remoção como o PVA e mergulhá-lo repetidamente na resina, à semelhança do processo de fabrico de velas, sendo o número de imersões definido pela espessura pretendida para o modelo positivo. No final, bastaria dissolver o molde negativo em água:

- Vantagens: processo de produção muito simples;
- Desvantagens: o material do molde negativo tem um custo elevado, sendo sempre perdido no processo. Depois de impresso, o modelo, já sem qualquer rigidez estrutural própria, seria também difícil de fixar à estrutura de suporte dos manipuladores, exigindo peças de fixação dedicadas e de integração complexa.

Por estas razões, optou-se por manter o método de impressão em peça única com suportes exteriores.

3.2.6 Evolução do processo de impressão

Inicialmente, os suportes eram impressos no mesmo material do modelo, devido às limitações da maquinaria disponível na altura, com apenas um bico de impressão. Este método funcionava razoavelmente bem com filamentos de dureza elevada, como o 95A, mas mostrava-se bastante problemático com materiais mais moles, como o Filaflex TPE 70A. A mesma característica que tornava estes materiais interessantes para o projeto tornava-os também problemáticos. Ao usar o mesmo material para o suporte e para o objeto, toda a impressão se comportava como um único e grande objeto mole, cuja vibração aumentava com a altura já impressa, entrando em ressonância com o próprio movimento

da cabeça de impressão e criando grandes inconsistências. Por frações de milissegundo e de milímetro, a posição da cabeça de impressão deixava de corresponder à posição real da última camada impressa. (vou adicionar imagens sobre isto nesta secção)

Uma das soluções equacionadas para este problema passava por substituir o filamento por um material especial de dureza variável, com um agente expansivo interno que utiliza o próprio calor do bico como catalisador, permitindo teoricamente atingir durezas entre 80A e 50A consoante a temperatura, segundo a informação disponibilizada pela marca. O único inconveniente deste material seriam os resíduos gerados pelo agente expansivo, que poderiam comprometer a qualidade de impressão, ainda que de forma mitigável. Esta solução acabou por nunca ser necessária, pois, sensivelmente na mesma altura, a universidade adquiriu três novas impressoras 3D, entre elas uma Bambu Lab H2D, equipada com dois bicos independentes que permitem imprimir, em simultâneo, materiais com propriedades e compatibilidades distintas, uma aquisição que resolveu o problema.

Passou-se então a um método que mantinha o TPE 70A como material principal do modelo, mas com PLA como material de suporte, funcionando simultaneamente como suporte estrutural e como suporte de amortecimento de vibração para o modelo principal. (vou adicionar uma imagem sobre isto)

Este método trouxe uma melhoria enorme na qualidade de impressão, mas tornou mais visíveis outros problemas. Como já referido, os materiais flexíveis de dureza muito baixa são especialmente difíceis de imprimir com fiabilidade. Este caso não foi exceção. Detetaram-se, durante e após a impressão, inconsistências de aderência entre camadas e entre paredes do objeto, camadas inteiras com falta de material, tornando-as propensas a quebrar ou rasgar e, o pior de todos os problemas, uma quantidade excessiva de stringing no interior do modelo, precisamente a zona mais importante para este trabalho. Isto persistia mesmo após inúmeras calibrações dos parâmetros de retração, da velocidade de impressão, da temperatura de impressão, do Pressure Advance e dos níveis de humidade do filamento. O Pressure Advance é uma funcionalidade do firmware do slicer que ajusta antecipadamente a pressão de extrusão nas mudanças de velocidade, reduzindo o babar de material e o stringing nos cantos. O TPU é, além disso, frequentemente descrito como uma “esponja”, por ser um material higroscópico, ou seja, por absorver facilmente humidade do ar, sendo por isso necessário secá-lo constantemente e mantê-lo selado o máximo de tempo possível.

Foram também testadas técnicas e opções internas do slicer para mitigar o stringing, como alterar a localização da costura das paredes para uma zona menos visível, calibrar constantemente os valores de transição entre paredes, modificar as opções de geometria de retração e forçar as deslocações da cabeça de impressão a ocorrerem, sempre que possível, sobre as zonas de enchimento.

Ainda assim, nada resolvia de forma satisfatória o problema do stringing, nem

mesmo técnicas de limpeza manual, como escovilhões para aquários, jatos de ar comprimido ou a queima das próprias linhas no interior do modelo. Algumas destas técnicas conseguiam reduzir a visibilidade do problema nas ramificações mais próximas da entrada, mas nas ramificações mais profundas era praticamente impossível.

A única forma de resolver, ou melhor, de mitigar este problema, foi escolher outro material como filamento flexível principal, o TPU MorPhlex 95A/75A, da empresa BiQU. Ao contrário dos outros filamentos de dureza variável considerados, apresenta uma transição de dureza fixa, começando com uma dureza de 95A antes de ser impresso e estabilizando permanentemente numa dureza de $75A \pm 3A$ depois de impresso e aquecido. Esta característica torna-o muito mais estável, fiável e seguro do que as alternativas anteriores, permitindo, com a calibração correta, atingir uma qualidade de impressão quase perfeita e praticamente sem resíduos. Um outro ponto forte deste filamento é a sua cor, exclusiva da BiQU, muito próxima da cor real do interior do pulmão, o que o tornava ainda mais adequado a este trabalho.

Este material revelou, no entanto, uma deformação predominantemente plástica em vez de elástica quando sujeito a grandes esforços repetidos, como as contrações e expansões constantes que a atuação ativa das paredes exigiria, uma característica que viria a pesar mais tarde na avaliação da viabilidade de atuadores dedicados a essa deformação, discutida na section 3.4.

3.3 Seleção do sistema de atuação

A definição do sistema de atuação foi uma das partes que mais evoluiu durante o projeto, sobretudo nas fases iniciais. A descrição dos movimentos pretendidos ainda não estava completamente consolidada e, por isso, foram exploradas soluções que mais tarde se revelaram pouco adequadas ao comportamento real que se pretendia reproduzir. Esta incerteza levou à análise de alternativas bastante diferentes entre si, desde motores de vibração, pensados para criar zonas de pressão localizada que pudessem sugerir o batimento cardíaco ou pequenas compressões das vias aéreas, até micro motores de passo, considerados para apertar gradualmente as paredes do modelo e simular constrangimentos associados ao ciclo respiratório.

Com a evolução do modelo da árvore brônquica e após a primeira reunião de acompanhamento com a equipa médica, tornou-se mais clara a natureza dos movimentos a implementar. O objetivo principal deixou de ser a criação de deformações locais independentes e passou a ser a reprodução controlada de deslocações globais e perturbações visíveis, associadas à respiração, ao batimento cardíaco e à tosse. Esta clarificação obrigou a rever a arquitetura de atuação inicialmente imaginada, baseada na distribuição de vários atuadores ao longo da árvore brônquica. Embora essa solução pudesse parecer direta para movimentos localizados, acabaria por tornar o sistema pesado do ponto de vista mecânico,

aumentando o número de motores, exigindo uma estrutura de suporte muito próxima do modelo e introduzindo dificuldades adicionais de montagem, calibração, sincronização e manutenção.

A solução adotada passou, por isso, pela utilização de manipuladores robóticos capazes de posicionar uma plataforma móvel dentro de um volume de trabalho definido por coordenadas cartesianas. Nesta abordagem, o movimento deixa de depender de vários atuadores distribuídos pela árvore brônquica e passa a ser gerado a partir da trajetória imposta à plataforma. A complexidade é, assim, transferida para a descrição matemática do movimento, em particular para o cálculo da cinemática inversa, permitindo alterar perfis de respiração, batimento cardíaco ou tosse sobretudo por software, sem obrigar a modificar fisicamente a disposição dos motores.

Dentro desta lógica, foram estudadas diferentes configurações de manipuladores delta, uma vez que este tipo de robô paralelo utiliza vários braços ligados a uma plataforma móvel comum. Ao contrário de um manipulador serial, em que cada junta depende diretamente da anterior, num manipulador delta os braços trabalham em conjunto para posicionar a plataforma no espaço. Esta geometria permite obter movimentos rápidos, repetíveis e compactos, mantendo os atuadores fixos na base e reduzindo a massa em movimento. Para este projeto, estas características eram particularmente interessantes, pois permitiam atuar sobre a árvore brônquica sem instalar motores diretamente sobre o modelo.

Foram considerados dois tipos principais de solução. A primeira recorria a seis atuadores, dois por braço, permitindo controlar a translação da plataforma nos eixos X , Y e Z e, adicionalmente, a sua orientação através de três graus de rotação. Esta configuração oferecia maior liberdade de movimento, mas introduzia uma complexidade mecânica e matemática superior à necessária para os fenómenos definidos para o protótipo. A segunda opção, posteriormente escolhida, utiliza apenas três atuadores, um por braço, controlando a posição da plataforma nos três eixos cartesianos. Apesar de não permitir comandar diretamente a rotação da plataforma, esta solução revelou-se suficiente para representar os deslocamentos pretendidos, simplificando a montagem, reduzindo o número de componentes e tornando mais direta a implementação da cinemática inversa.

Com esta decisão, a seleção dos atuadores passou a privilegiar servo-motores compactos, acessíveis e fáceis de controlar por PWM, tendo sido escolhidos os MG90S para a construção dos manipuladores. Esta escolha foi também reforçada pela comparação direta com motores de passo, avaliados em paralelo durante os testes iniciais de eletrónica. Apesar de os motores de passo garantirem maior capacidade de carga, a sua natureza de controlo em malha aberta obriga a uma rotina de homing sempre que o sistema é ligado, para que o motor recupere a noção da sua posição real, enquanto um servo-motor recebe apenas, através do sinal PWM enviado num único sentido, a posição pretendida, cabendo ao seu controlador interno ajustar automaticamente, através de uma malha de controlo

PID, o erro entre a posição atual e essa posição pretendida, até a atingir. Esta diferença simplifica significativamente o arranque e o funcionamento do sistema. A escolha destes servos foi ainda coerente com o objetivo de manter o sistema simples e repetível, concentrando a coordenação dos movimentos no firmware e no cálculo das posições da plataforma. Antes da montagem final, foram realizados testes de encaixe entre os MG90S e as peças impressas, verificando tolerâncias, alinhamento da patilha do servo e liberdade de rotação dos braços. Estes ensaios foram acompanhados por um processo gradual de calibração, no qual se confirmou a posição neutra de cada servo, os limites angulares admissíveis e a correspondência entre o comando PWM enviado pelo firmware e o movimento real observado no manipulador.

Esta escolha mecânica foi acompanhada, ainda no final de abril de 2026, pelo início do desenvolvimento do código de cinemática inversa associado a este tipo de manipulador, descrito em detalhe na section 4.6. Ao longo deste processo, a validação do sistema foi feita de forma cruzada: o comportamento físico dos braços montados em bancada era comparado com as posições calculadas pelo firmware, permitindo ajustar limites, corrigir desvios e garantir que o movimento matematicamente previsto podia ser reproduzido de forma segura pela estrutura real.

3.4 Eletrónica de controlo e alimentação

O desenvolvimento da eletrónica teve início a meio do desenvolvimento do hardware, depois de confirmada a viabilidade de impressão do modelo e assim que os componentes eletrónicos chegaram, encontrando-se este passo diretamente ligado à implementação dos movimentos, uma vez que, sem eletrónica funcional, não era possível testar sensores, microcontroladores nem motores. A arquitetura prevista separa a interface de utilizador e o cálculo de alto nível do controlo direto dos atuadores, cabendo ao Raspberry Pi 4B a responsabilidade pela HMI, pela monitorização e pelo envio de comandos ou posições, enquanto o ESP32-S3 atua como microcontrolador de tempo real, recebendo dados por comunicação série e convertendo-os em comandos para os servos.

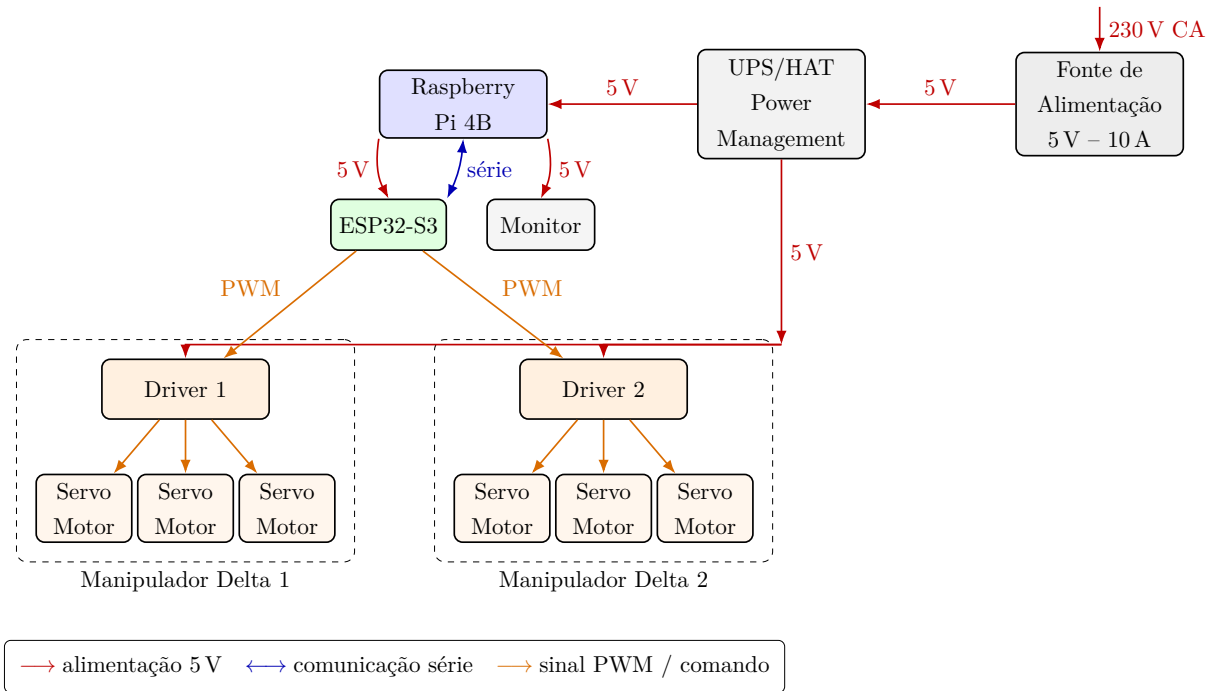


Figura 3.3: Esquema consolidado de alimentação, comunicação e comando dos dois manipuladores delta.

3.4.1 Testes iniciais de eletrônica

Os primeiros testes de eletrônica focaram-se exclusivamente na validação individual de cada componente, antes de qualquer tentativa de integração. Começou por se confirmar uma alimentação estável e a comunicação com o ESP32, verificando se este arrancava corretamente e permitia programação, sendo nesta fase alimentado apenas pela porta série do computador, uma solução pensada como temporária mas que viria a manter-se, de forma incidental, durante grande parte do desenvolvimento. (vou adicionar aqui os esquemas elétricos de cada teste individual)

Em seguida, testaram-se isoladamente os três tipos de atuadores considerados nesta fase, confirmando apenas que cada um podia ser ligado e posto em movimento. O motor de vibração foi ligado diretamente ao ESP32, por se esperar um consumo reduzido, enquanto o servo-motor e o micro motor de passo foram alimentados a partir de uma fonte externa regulável, com limite de corrente definido em 2 A. O servo foi testado a 6 V, próximo do limite superior do intervalo de funcionamento especificado pelo fabricante para o MG90S, entre 4,8 V e 6 V, com o objetivo de obter uma velocidade de resposta favorável durante os testes de deformação localizada e concentrada das paredes do modelo, então ainda em avaliação. Já o motor de passo foi testado com um driver dedicado, alimentado a 9 V, o valor mínimo disponível acima dos 5 V na mesma fonte regulável, que permitia um intervalo de saída entre os 3 V e os 9 V.

3.4.2 Arquitetura de alimentação e avaliação de motores de passo

Foi por esta altura que se consolidou a escolha do manipulador delta e dos servo-motores como atuador principal, pelas razões detalhadas na section 3.3, mantendo-se contudo, nesta fase do projeto, a hipótese de recorrer também a motores de passo para reproduzir movimentos de deformação e contração localizada das vias aéreas. Os esquemas iniciais de alimentação previam por isso dois níveis de tensão distintos. Um dos níveis destinava-se ao microcontrolador, aos servos e aos sensores, enquanto o outro seria dedicado exclusivamente aos motores de passo, tendo sido equacionadas duas arquiteturas possíveis para o obter:

- Partir de uma fonte de 12 V, alimentando diretamente os drivers dos motores de passo e reduzindo essa tensão através de um conversor step-down para os 5 V do restante sistema;
- Partir de uma fonte de 5 V e elevar a tensão através de conversores step-up dedicados aos drivers dos motores de passo, uma opção descartada devido ao consumo de potência e às perdas energéticas associadas a este tipo de conversão.

Por exclusão, a primeira arquitetura, baseada numa fonte de 12 V e num conversor step-down, tornou-se a opção mais viável.

Foi também nesta fase que surgiu a decisão de acrescentar uma interface dedicada ao sistema, inicialmente pensada como um pequeno ecrã OLED mas rapidamente substituída pela solução final, um Raspberry Pi associado a um monitor integrado. Esta escolha trouxe consigo a necessidade de proteger o Raspberry Pi de perdas de alimentação inesperadas, um cuidado já conhecido de experiência anterior, uma vez que a perda de energia durante a escrita no cartão SD onde reside o sistema operativo pode corromper esse cartão ou, em casos mais graves, danificá-lo de forma permanente. Como o Raspberry Pi é tipicamente alimentado através de uma porta de 5 V e 3 A, esta necessidade era compatível com a arquitetura de 12 V e conversor step-down já equacionada para os motores de passo.

3.4.3 Dimensionamento da fonte de alimentação

O dimensionamento da fonte teve por base o cenário então previsto de três manipuladores delta, correspondendo a nove servos MG90S. Este servo apresenta um consumo nominal entre 150 e 250 mA, mas pode atingir cerca de 700 mA em condição de bloqueio (stall), pelo que, ainda que fosse pouco provável que todos os nove servos entrassem em stall em simultâneo, se optou por dimensionar sempre para o pior caso possível, de forma a garantir o funcionamento do sistema mesmo nas condições mais exigentes, resultando este valor de

bloqueio numa corrente total próxima dos 6,3 A para os nove servos, contra apenas 2,6 A em regime nominal. Os motores de passo, por sua vez, foram teorizados num total de seis unidades distribuídas pelo modelo, cada uma com um consumo máximo de 150 mA, o que acrescentava cerca de 900 mA ao total. Somando ainda o consumo do Raspberry Pi, estimado em 3 A, o cenário de 12 V resultava numa corrente total próxima dos 10,5 A, sendo expectável que, com recurso a um conversor step-down, a corrente efetivamente exigida à fonte pelos servos e pelo Raspberry Pi fosse consideravelmente menor do que este valor calculado diretamente a 5 V.

Esta arquitetura acabou, no entanto, por não avançar. Após várias discussões, concluiu-se que a utilização de motores de passo para reproduzir contrações e dilatações localizadas das vias aéreas seria uma implementação de elevada complexidade, sobretudo devido ao comportamento do filamento MorPhlex então em uso, que revelava uma deformação predominantemente plástica em vez de elástica sob grandes esforços repetidos, como as contrações e expansões constantes que este tipo de atuação exigiria, conforme já referido na section 3.2.6. Com a exclusão dos motores de passo, deixou de haver justificação para manter os dois níveis de tensão e para a fonte de 12 V, tendo o sistema sido simplificado para um único nível de 5 V, com uma corrente total estimada, para o pior caso possível, de cerca de 9,6 A. Foi por isso escolhida uma fonte de alimentação de 5 V e 10 A, com margem suficiente acima do valor estimado.

Com a fonte selecionada, foram realizados testes com os dois manipuladores a funcionar em simultâneo, tendo-se observado uma pequena variação de tensão, entre 100 e 200 mV, associada aos picos de corrente na comutação dos servos. Ainda que este valor não pareça elevado, procurou-se minimizá-lo ao máximo, uma vez que o Raspberry Pi é particularmente sensível a variações na sua alimentação, podendo sofrer reinícios inesperados mesmo com uma placa UPS a atuar como filtro adicional contra este tipo de perturbação. A pesquisa realizada confirmou ser prática comum a colocação de condensadores de desacoplamento junto à alimentação de cada servo, para atenuar precisamente este tipo de queda de tensão transitória, com a documentação e os fóruns consultados a recomendarem valores tipicamente entre os 230 e os 700 μF . Optou-se por condensadores de 470 μF junto de cada servo. (vou adicionar imagens sobre isto)

3.4.4 Gestão de energia do Raspberry Pi

Mais tarde, o projeto passou de três para apenas dois manipuladores delta, reduzindo a complexidade face aos resultados pretendidos e diminuindo ainda mais a carga total sobre a fonte de alimentação. Em contrapartida, foi selecionada a UPS/HAT Suptronics X1209, que dispõe de duas entradas de alimentação distintas, uma de 5 V e 5 A e outra entre os 6 V e os 18 V até 3 A, uma flexibilidade que permitiu continuar a utilizar a mesma fonte de 5 V e 10 A já escolhida anteriormente. Ao contrário de grande parte das alternativas

pesquisadas, que ou se limitavam a placas de gestão de energia bastante dispendiosas, ou eram placas de bateria incapazes de assegurar uma transição fiável entre a alimentação de rede e a alimentação por bateria, por serem pensadas apenas para funcionar a partir da bateria, esta HAT combina simultaneamente as funções de UPS e de gestão de energia, o que justificou a sua escolha. Foi associada a uma bateria LiPo de 2100 mA h e 3,7 V.

3.4.5 Da protoboard à PCB

Depois de concluídos os testes do circuito numa protoboard, o mesmo foi transferido para uma PCB pré-furada, onde foram implementados os drivers referidos anteriormente, um por cada manipulador delta, cada um responsável por controlar os três servos correspondentes. Foi também montada e soldada uma PCB própria para alojar o microcontrolador ESP32-S3. Cada driver recebe os comandos enviados pelo ESP32 e a alimentação diretamente da fonte principal, distribuída através de um barramento comum a cada um dos três servos, sempre por intermédio do respetivo condensador de desacoplamento. O ESP32, por sua vez, pode ser alimentado pelo Raspberry Pi através de USB-C, mantendo simultaneamente a ligação série necessária à comunicação.

Tabela 3.1: Componentes principais previstos para o sistema.

Componente	Função	Observações
Raspberry Pi 4B	Interface e coordenação	Envia comandos ou posições para o ESP32
ESP32-S3	Controlo dos atuadores	Executa FSMs e gera comandos para servos
Servo-motores	Atuação mecânica	Seis unidades distribuídas pelos dois manipuladores delta
Driver Servo-motores	Interface de potência/comando	Cada driver comanda três servo-motores de um manipulador delta
UPS/HAT	Gestão de energia	Alimentação do Raspberry Pi e transição para bateria
Fonte de Alimentação 5 V–10 A	Alimentação principal	Recebe 230 V e fornece 5 V ao sistema
LiPo Battery 2100 mA 3,7 V	Alimentação de reserva	Bateria associada ao sistema UPS/HAT

3.5 Integração mecânica e segurança

A integração mecânica deve garantir que o movimento é transmitido à árvore brônquica sem esforços excessivos, folgas significativas ou interferência com a passagem do broncoscópio, devendo a estrutura permitir ainda acesso para manutenção, limpeza e substituição de peças. Do ponto de vista de segurança, os limites de curso do manipulador

e os limites angulares dos servos devem ser definidos no firmware, evitando posições impossíveis ou capazes de danificar o modelo.

A fase de montagem começou pela impressão das peças estruturais em PLA preto, incluindo a base circular, os braços superiores, os elos de ligação e a plataforma móvel central. Numa primeira fase, a estrutura foi montada sem servos para verificar folgas, alinhamento das articulações e comportamento geométrico do conjunto (figura 3.4), etapa que permitiu identificar zonas onde era necessário rever tolerâncias e confirmar que os braços articulados rodavam sem bloqueio.



Figura 3.4: Estrutura mecânica de um manipulador delta em PLA preto, antes da instalação dos servos, maio de 2026. A base circular e os três braços articulados estão já montados.

A instalação dos servos foi feita de forma gradual, com validação individual de cada canal, tendo os dois manipuladores sido testados, numa fase intermédia, com ligações diretas ao ESP32 sem PCB de interface, usando jumpers para confirmar o funcionamento elétrico básico antes de qualquer ligação definitiva. A figura 3.5 mostra os dois manipuladores montados com todos os servos instalados, prontos para a fase de calibração de firmware.

Para acomodar o Raspberry Pi 4B e a fonte de alimentação de forma compacta e robusta, foi projetada e impressa uma caixa em PLA branco. Esta estrutura, visível na figura 3.6, agrupa o Raspberry Pi com a fonte de alimentação comutada numa única unidade que pode ser posicionada independentemente do modelo dinâmico.



Figura 3.5: Os dois manipuladores delta montados com os servos Waveshare instalados, prontos para calibração de firmware, junho de 2026. Os cabos dos servos são visíveis mas ainda sem organização final de cablagem.

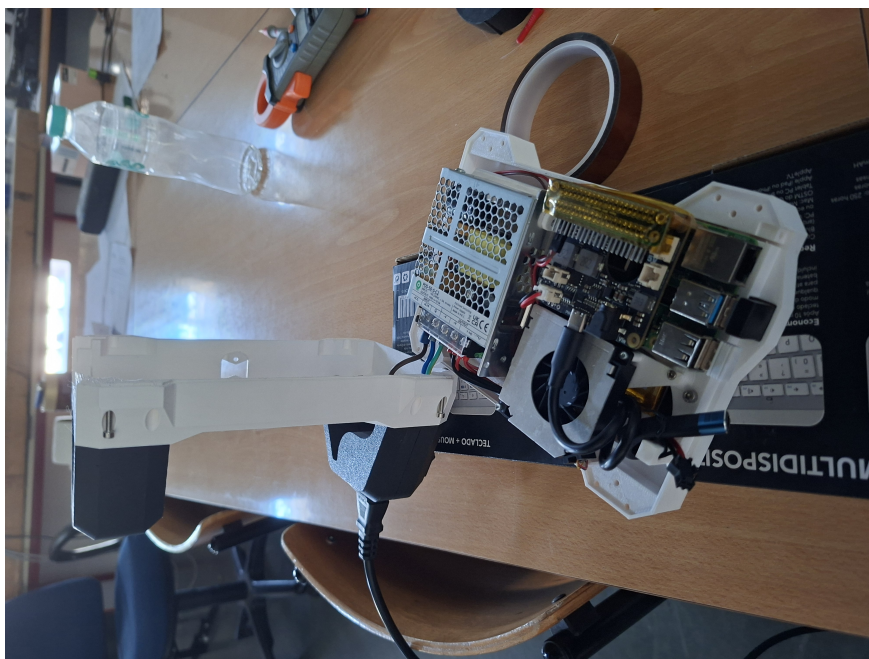


Figura 3.6: Caixa impressa em PLA branco que aloja o Raspberry Pi 4B e a fonte de alimentação comutada, junho de 2026.

Capítulo 4

Desenvolvimentos do Software e Configurações

O software foi desenvolvido em paralelo com o hardware, mas em camadas progressivas. Começou por testes elementares de comunicação série para confirmar que o ESP32 e o Raspberry Pi conseguiam trocar dados de forma fiável. A partir daí foram adicionados, um a um, os blocos de geração de trajetórias, de máquinas de estados, de cinemática inversa e de interface gráfica. Este capítulo descreve essa construção por partes, da arquitetura global até aos detalhes de calibração e sincronização, seguindo a ordem em que cada bloco foi desenvolvido e validado.

4.1 Arquitetura geral do software e ambiente de desenvolvimento

O software foi organizado para separar a definição dos movimentos, o estado de execução, a comunicação série, a cinemática inversa, o acionamento dos servos e a interface HMI. Esta separação permite que os mesmos mecanismos de execução sejam usados para diferentes fenómenos fisiológicos, como respiração, batimento cardíaco e tosse, mantendo a interface de utilizador independente da lógica de baixo nível executada no ESP32-S3.

A arquitetura global, representada na figura 4.1, organiza o controlo em blocos independentes de HMI, comunicação, interpretação de comandos, atualização de movimento e cálculo cinemático, onde o Raspberry Pi executa a interface gráfica e envia comandos por comunicação série, enquanto o ESP32-S3 lê esses dados, interpreta comandos, atualiza estruturas de movimento, prepara eventos como a tosse e calcula as posições finais que serão convertidas pela cinemática inversa.

O firmware foi desenvolvido em C++ para a plataforma ESP32, usando o ecossistema Arduino/PlatformIO, recorrendo à biblioteca `ESP32Servo` para o controlo dos servos e à biblioteca `Adafruit_NeoPixel` para a indicação visual de estados por LED RGB.

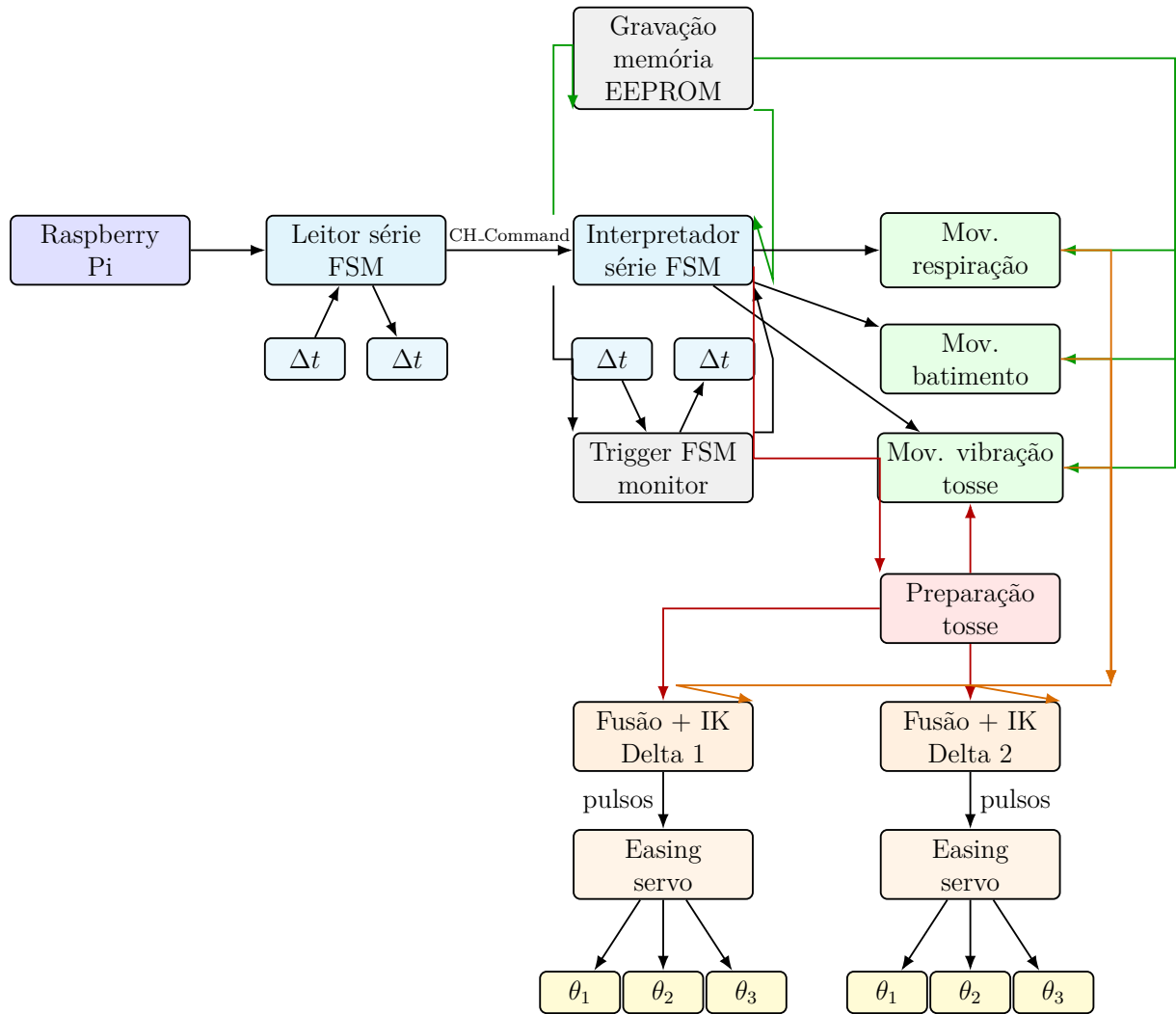


Figura 4.1: Estrutura global das máquinas de estados e blocos de controle.

O ponto de partida deste firmware foi um exemplo de referência para manipuladores delta, disponível publicamente, adaptado no final de abril de 2026 à geometria e aos servos usados neste projeto. Depois de confirmado o funcionamento básico da cinemática e do acionamento dos servos, esse código de referência foi progressivamente separado do código próprio do projeto, o que permitiu reescrever cada bloco — estruturas de dados, geração de movimento, máquinas de estados — de forma independente, sem manter a organização original do exemplo.

O ESP32 comunica com o computador ou com o Raspberry Pi 4B por porta série a 115200 baud, tendo sido também usados, durante o desenvolvimento, scripts Python para leitura da porta série, depuração de mensagens e validação de comandos enviados pelo microcontrolador.

Antes da construção da HMI final, foram criados testes simples de comunicação série em Python. Estes testes permitiam abrir a porta série, controlar os sinais DTR/RTS, ler mensagens enviadas pelo ESP32 e introduzir comandos manualmente no terminal. Esta etapa foi importante para confirmar que o backbone de comunicação entre o computador/Raspberry Pi e o ESP32 estava funcional antes de se acrescentar uma interface gráfica. Em paralelo, foram usados códigos de cálculo de movimentos para validar a geração de pontos, curvas e gráficos que mais tarde passariam a ser integrados na interface. Estes scripts de comunicação e de cálculo foram desenvolvidos e validados de forma isolada, ainda durante maio de 2026, antes de qualquer integração com uma interface gráfica.

A figura 4.2 mostra o ambiente de trabalho típico desta fase: o portátil com o PlatformIO aberto a lado de um osciloscópio RIGOL utilizado para verificar os sinais PWM, o ESP32 montado em breadboard com jumpers para os servos, e uma tablet com os diagramas das máquinas de estados usados como referência durante o desenvolvimento.

4.2 Movimentos representados no protótipo

O estado da arte identificou vários movimentos observáveis durante a broncoscopia. No entanto, para a primeira versão funcional do modelo, foram selecionados apenas os movimentos com melhor relação entre relevância clínica, viabilidade mecânica e simplicidade de controlo: respiração, batimento cardíaco e tosse. Os valores de frequência e amplitude usados nas secções seguintes, resumidos na tabela 4.1, foram retirados diretamente do documento de trabalho fornecido pela equipa de Pneumologia do CHUA [28].

A respiração foi definida como um movimento cíclico, com frequência aproximada de 12 a 20 ciclos por minuto e deslocamento predominantemente craniocaudal, devendo a implementação permitir pequenas irregularidades e alteração de amplitude, para que o movimento não pareça completamente artificial.

O batimento cardíaco foi associado principalmente ao brônquio principal esquerdo, devido à proximidade anatômica do coração, prevendo-se um movimento de baixa ampli-

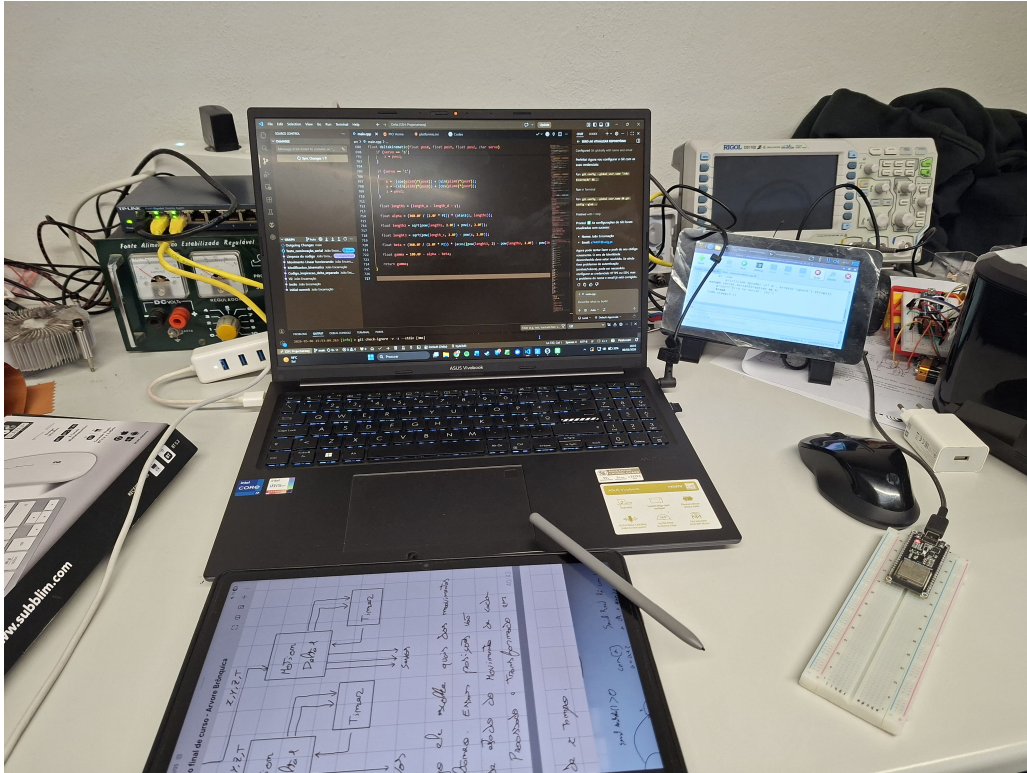


Figura 4.2: Ambiente de desenvolvimento de software e eletrônica: portátil com PlatformIO, osciloscópio RIGOL, ESP32 em breadboard e tablet com diagramas de apoio, maio de 2026.

tude, cerca de 1 a 3 mm, com frequência aproximada de 60 a 120 batimentos por minuto. No protótipo, este movimento é tratado como uma oscilação sobreposta ao movimento respiratório.

A tosse foi caracterizada como um evento abrupto, irregular e de maior amplitude relativa e, em vez de ser apenas uma trajetória independente, deve alterar temporariamente o comportamento respiratório, aumentando a amplitude e introduzindo uma perturbação rápida. Movimentos como deformação ativa do lúmen, estenose localizada e resistência ao contacto com a sonda não foram incluídos nesta versão, por exigirem atuadores distribuídos e materiais com características mecânicas mais específicas.

Tabela 4.1: Movimentos considerados para a primeira versão do protótipo.

Movimento	Frequência	Amplitude pre- vista	Observações
Respiração	12–20 ciclos/min	5–10 mm	Cíclico, com pequenas irregularidades
Batimento cardíaco	60–120 bpm	1–3 mm	Oscilação de baixa amplitude
Tosse	Evento curto	5–25 mm	Abrupto, não linear e irregular

4.3 Geração de trajetórias (Python/Raspberry Pi)

Foram definidos três perfis principais, procurando transformar a descrição qualitativa dos movimentos observáveis durante a broncoscopia em trajetórias discretas que pudessem ser executadas pelo modelo: a respiração foi descrita como um movimento lento, cíclico e dominante, associado à expansão e relaxamento do conjunto; o batimento cardíaco, por sua vez, foi tratado como uma perturbação de menor amplitude e maior frequência, sobreposta ao movimento principal; e a tosse foi interpretada como um evento curto e abrupto, composto por uma alteração rápida da trajetória respiratória e por uma vibração localizada de curta duração.

No Raspberry Pi, a parte escrita em Python tem uma função diferente: transformar parâmetros configuráveis em curvas, pontos e gráficos para a HMI. Este cálculo foi desenvolvido inicialmente fora da interface, para validar as curvas, os pontos e as coordenadas antes de os integrar na aplicação. Na versão para Raspberry Pi, o cálculo é feito pelo módulo `calculo_movimentos.py`. A interface permite alterar parâmetros geométricos, número de pontos e parâmetros temporais; depois, o módulo calcula os pontos de passagem e devolve gráficos 2D e 3D para visualização. Estes pontos são guardados como coordenadas X , Y e Z , podendo servir de base para atualizar posteriormente as estruturas de movimento usadas pelo firmware.

Esta lógica de cálculo evoluiu por etapas ao longo de maio de 2026: a versão inicial, a meio do mês, gerava apenas uma curva padrão genérica com suavização interna por *easing*; poucos dias depois, o código evoluiu para as três curvas específicas atualmente usadas — elipse para o batimento, parábola para a respiração e circunferência para a tosse — já preparadas para alimentar a etapa de fusão dos movimentos e a cinemática inversa dos dois manipuladores.

De forma geral, cada movimento é definido por uma curva contínua $\mathbf{p}(u)$ e por um conjunto discreto de N pontos:

$$\mathbf{p}_i = \begin{bmatrix} X_i \\ Y_i \\ Z_i \end{bmatrix} = \mathbf{p}(u_i), \quad i = 0, 1, \dots, N - 1. \quad (4.1)$$

O batimento cardíaco foi representado por uma elipse rotacionada no plano XY , uma escolha que permite produzir uma perturbação fechada, suave e de pequena amplitude, coerente com a ideia de oscilação periódica provocada pelo batimento, e que é

definida por:

$$X(u) = x_c + a \cos(u) \cos(\alpha) - b \sin(u) \sin(\alpha), \quad (4.2)$$

$$Y(u) = y_c + a \cos(u) \sin(\alpha) + b \sin(u) \cos(\alpha), \quad (4.3)$$

$$Z(u) = 0, \quad (4.4)$$

em que a e b são os semi-eixos da elipse, (x_c, y_c) é o centro da curva e α é o ângulo de rotação, podendo o sentido de percurso ser horário ou anti-horário. No conjunto de parâmetros usado nos testes da interface, foram considerados $a = 5 \text{ mm}$, $b = 2 \text{ mm}$, $x_c = y_c = 3,6 \text{ mm}$, $\alpha = 45^\circ$, $N = 10$ pontos e sentido horário.

Para a respiração foi usada uma parábola no plano ZX , por ser uma forma simples de representar uma subida progressiva com variação lenta no início e mais acentuada no final do deslocamento. Esta curva permite simular um movimento respiratório principal, controlando diretamente a altura máxima e a largura da trajetória. A mesma família de curva também pode ser usada como deslocamento rápido associado à tosse, alterando sobretudo o período e a forma como o evento é ativado. A curva base é:

$$X(u) = u, \quad (4.5)$$

$$Z(u) = \frac{H}{L^2} u^2, \quad (4.6)$$

$$Y(u) = 0, \quad 0 \leq u \leq L, \quad (4.7)$$

em que H é a altura máxima e L é a largura máxima. Quando se pretende inclinar a curva no espaço, o valor $Z(u)$ pode ser projetado por uma rotação em torno do eixo X :

$$X_{3D}(u) = X(u), \quad (4.8)$$

$$Y_{3D}(u) = Z(u) \sin(\theta), \quad (4.9)$$

$$Z_{3D}(u) = Z(u) \cos(\theta). \quad (4.10)$$

Nos valores usados nos testes, foram considerados $H = 20 \text{ mm}$, $L = 12 \text{ mm}$, $N = 10$ pontos e $\theta = 0^\circ$, tendo sido usado, para a respiração, um período $T = 4 \text{ s}$, com pausa inicial de $0,6 \text{ s}$, pausa final de $0,1 \text{ s}$ e ponto inicial igual a 5, de modo a iniciar a trajetória a partir de uma posição intermédia.

Além da alteração do movimento respiratório, a tosse inclui uma vibração curta em torno da posição de referência. Esta vibração foi representada por uma circunferência no plano XY , por permitir uma perturbação fechada e rápida, sem deslocamento vertical

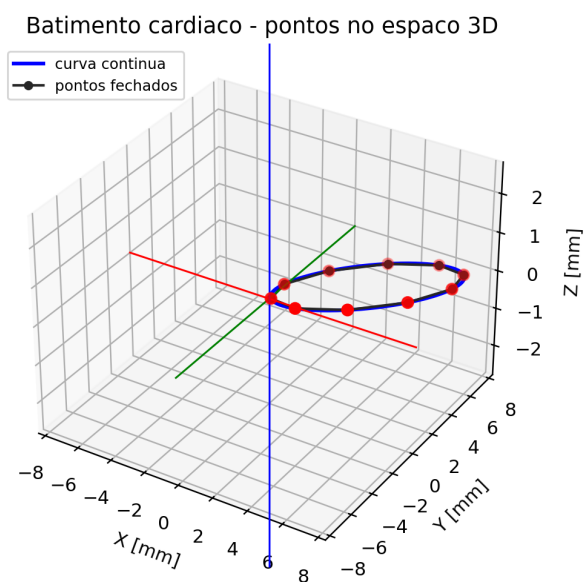
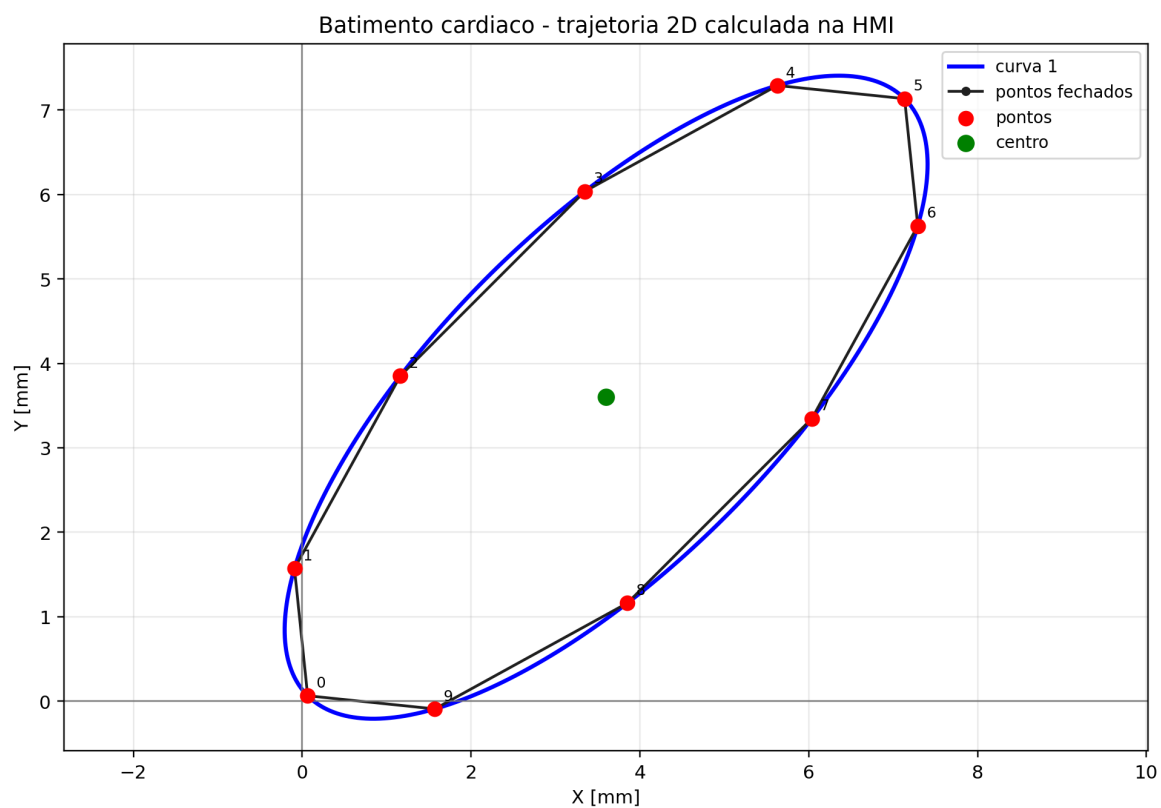


Figura 4.3: Trajetória calculada para o movimento de batimento cardíaco, com representação 2D e 3D dos pontos de passagem.

Respiracao

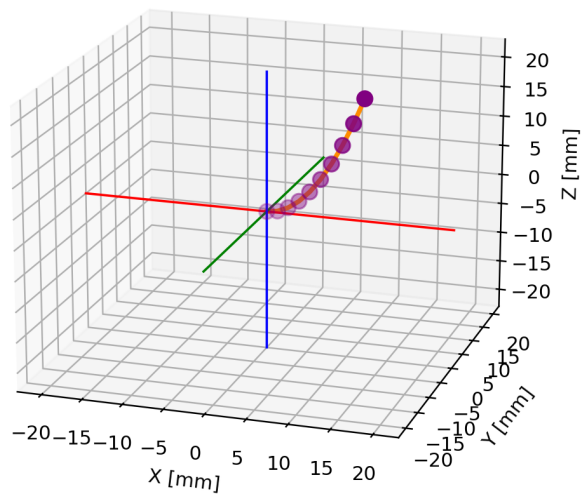
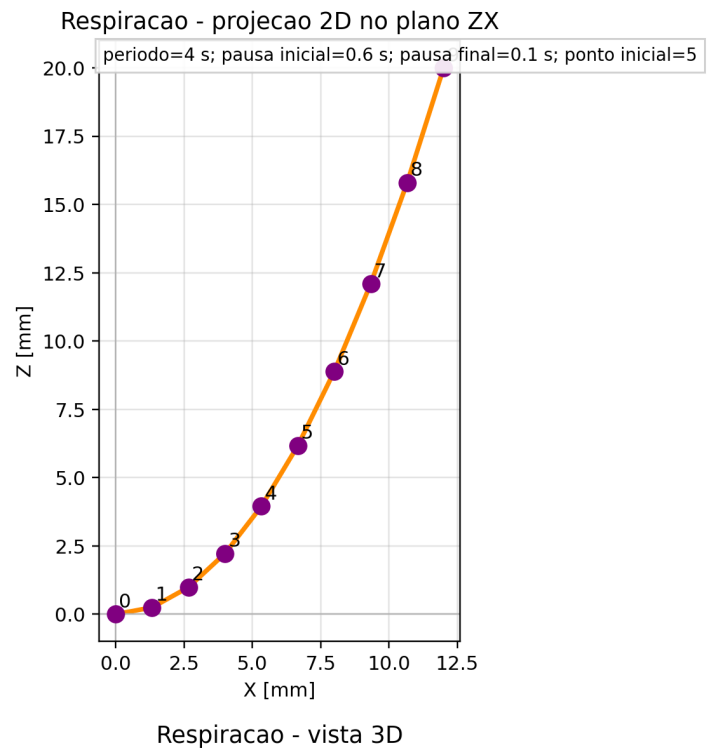


Figura 4.4: Trajetória calculada para o movimento respiratório, com representação 2D no plano ZX e visualização 3D.

adicional:

$$X(u) = r \cos(u), \quad (4.11)$$

$$Y(u) = r \sin(u), \quad (4.12)$$

$$Z(u) = 0. \quad (4.13)$$

O raio r controla a amplitude da vibração, o número de pontos controla a resolução da trajetória e o período define a rapidez do evento, tendo sido usado, no caso apresentado, $r = 2 \text{ mm}$, $N = 10$ pontos, sentido horário e período de $0,5 \text{ s}$.

Tabela 4.2: Parâmetros ajustáveis usados no cálculo dos movimentos na HMI.

Movimento	Parâmetros geométricos	Parâmetros temporais e de execução
Batimento cardíaco	a, b, x_c, y_c, α , número de pontos e sentido da elipse	Período, pausa inicial, pausa final e ponto inicial
Respiração	Altura H , largura L , ângulo θ e número de pontos da parábola	Período, pausa inicial, pausa final, ponto inicial e modo bidirecional
Vibração da tosse	Raio r , número de pontos e sentido da circunferência	Período, pausa inicial, pausa final e ponto inicial

(conteúdo adicional a enviar sobre o cálculo dos movimentos e pontos)

4.4 Estruturas de dados

A estrutura `Manipulador_Config` concentra os parâmetros geométricos e de calibração, entre os quais se encontram os comprimentos $L1$, $L2$ e $L3$, os offsets do servo nos eixos X e Z , os limites angulares e os valores mínimos e máximos de pulso para cada servo.

A estrutura `MotionStorage` guarda a definição de uma trajetória, incluindo arrays de pontos X , Y e Z para cada movimento, número de pontos, período total, tempos de pausa no início e no fim, índice inicial e informação sobre se o movimento é bidirecional; já a estrutura `MotionInstance` representa o estado dinâmico de uma trajetória em execução, guardando ponteiros, flags, índices, direção, tempos decorridos e estado atual da FSM.

4.5 Máquinas de estados e execução dos movimentos

No ESP32, o firmware não tem como função principal desenhar ou recalculas as curvas geométricas dos movimentos, mas sim gerir a execução, em tempo real, de pontos de trajetória já definidos, guardados em arrays de coordenadas X , Y e Z associados a uma estrutura `MotionStorage`, na qual se guardam ainda, para cada movimento, o número de

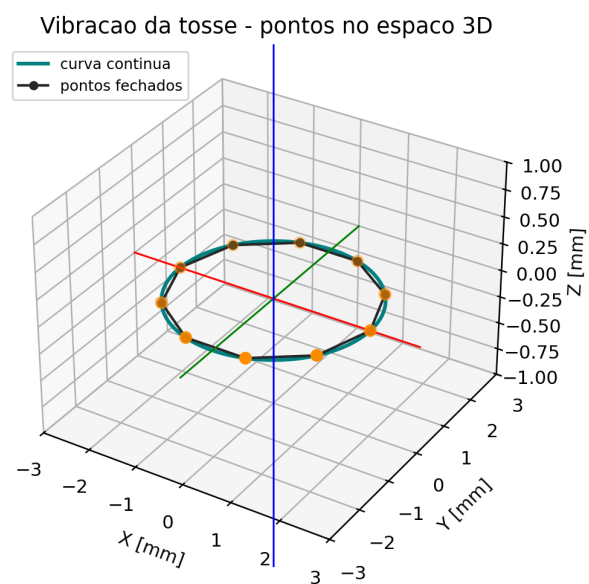
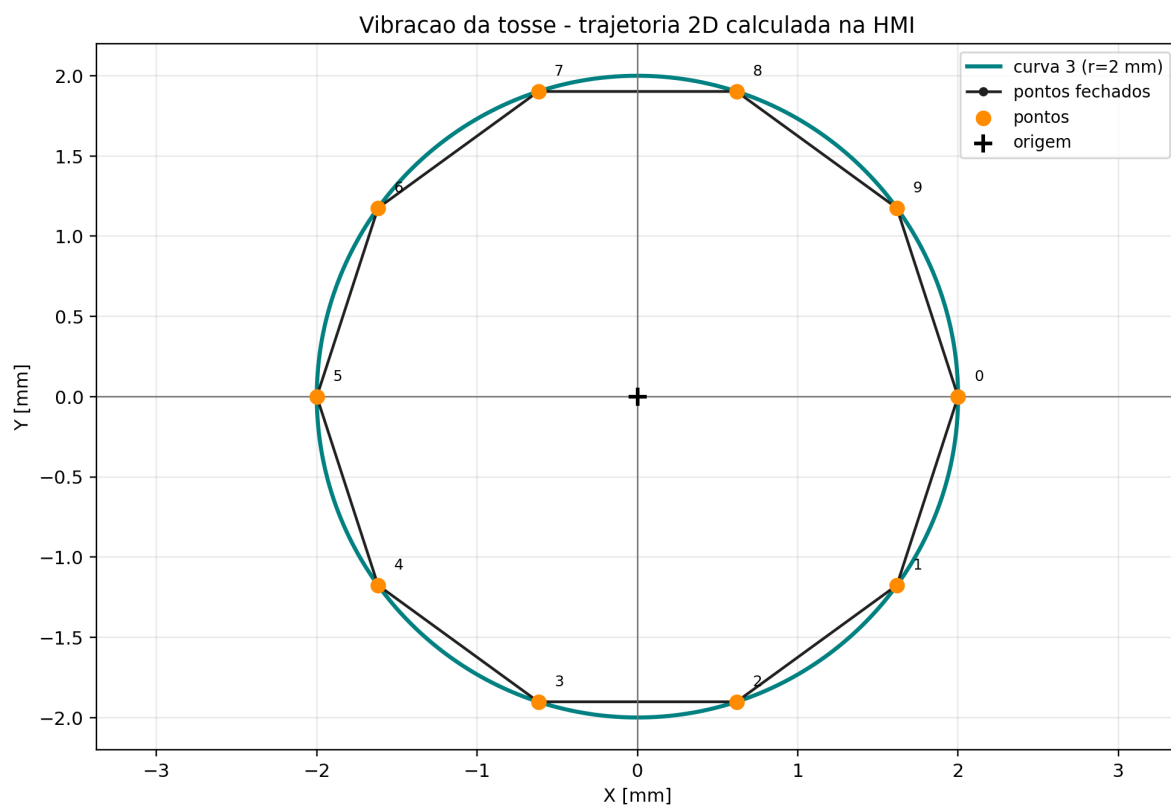


Figura 4.5: Trajetória calculada para a vibração associada à tosse, com representação 2D e 3D dos pontos de passagem.

Tabela 4.3: Pontos calculados para os movimentos usados nos testes da interface.

Ponto	Batimento cardíaco			Respiração			Vibração da tosse		
	X	Y	Z	X	Y	Z	X	Y	Z
0	0,06	0,06	0,00	0,00	0,00	0,00	2,00	0,00	0,00
1	-0,09	1,57	0,00	1,33	0,00	0,25	1,62	-1,18	0,00
2	1,16	3,85	0,00	2,67	0,00	0,99	0,62	-1,90	0,00
3	3,35	6,04	0,00	4,00	0,00	2,22	-0,62	-1,90	0,00
4	5,63	7,29	0,00	5,33	0,00	3,95	-1,62	-1,18	0,00
5	7,14	7,14	0,00	6,67	0,00	6,17	-2,00	0,00	0,00
6	7,29	5,63	0,00	8,00	0,00	8,89	-1,62	1,18	0,00
7	6,04	3,35	0,00	9,33	0,00	12,10	-0,62	1,90	0,00
8	3,85	1,16	0,00	10,67	0,00	15,80	0,62	1,90	0,00
9	1,57	-0,09	0,00	12,00	0,00	20,00	1,62	1,18	0,00

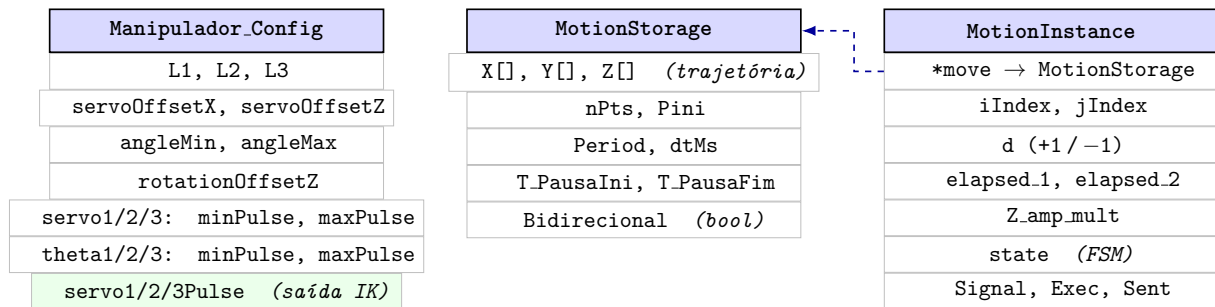


Figura 4.6: Relação entre as estruturas de dados principais do firmware.

pontos, o período total, as pausas no início e no fim, o ponto inicial e a indicação de o movimento ser, ou não, bidirecional.

Esta forma de execução, comum aos três movimentos, resulta de uma evolução ao longo de vários meses: a primeira versão do firmware, em abril de 2026, executava apenas um movimento linear fixo, usado para validar a cinemática inversa; em maio, esse movimento foi substituído por perfis com base na frequência do movimento e, por fim, pela lógica genérica de `MotionStorage/MotionInstance` aqui descrita, capaz de reutilizar o mesmo mecanismo de execução para qualquer trajetória.

A separação entre `MotionStorage` e `MotionInstance` permite distinguir a definição do movimento do seu estado de execução. A primeira estrutura contém os dados fixos da trajetória; a segunda guarda o estado temporário usado durante o funcionamento, incluindo índices atuais, direção de movimento, temporizadores e estado da máquina de estados. Assim, a mesma lógica de execução pode ser aplicada à respiração, ao batimento cardíaco e à vibração da tosse.

Durante a execução, a máquina de estados percorre a sequência de pontos e atualiza os índices i e j , que representam, respetivamente, o ponto de partida e o ponto de chegada do segmento atual. Entre estes dois pontos, a posição instantânea é obtida por interpolação linear:

$$\mathbf{p}(t) = (1 - \lambda) \mathbf{p}_i + \lambda \mathbf{p}_j, \quad \lambda = \frac{t - t_i}{\Delta t}, \quad 0 \leq \lambda \leq 1. \quad (4.14)$$

O intervalo temporal entre pontos é calculado a partir do período total do movimento. Para movimentos bidirecionais, como a respiração, considera-se a ida e o regresso sobre a mesma trajetória:

$$N_{\text{seg}} = \begin{cases} 2(N - 1), & \text{movimento bidirecional,} \\ N - 1, & \text{movimento não bidirecional,} \end{cases} \quad \Delta t = \frac{T}{N_{\text{seg}}}. \quad (4.15)$$

Depois de calculada a posição intermédia, essa coordenada segue para a etapa de fusão dos movimentos e para a cinemática inversa dos manipuladores delta, pelo que a lógica em C++ trata sobretudo da execução temporal, sincronização, pausas, inversão de direção e combinação dos movimentos no modelo dinâmico.

A função genérica de atualização de movimento percorre qualquer trajetória associada a uma `MotionInstance`. No desenvolvimento, esta rotina foi referida como `FSM_Motion_Update`. A figura 4.7 representa a sequência lógica desta máquina, mantendo os estados principais, as pausas inicial/final, a inversão de direção e o tratamento do movimento bidirecional.

A `FSM_Motion_Update()` recebe a cada ciclo de execução o sinal global de arranque `start`, a referência da instância de movimento `inst` e o instante de tempo atual `nowMs`.

A função não calcula posições — essa responsabilidade pertence à etapa seguinte — mas mantém atualizados os índices de trajetória e o cronómetro interno que a função de posição instantânea vai consumir.

A conversão da lógica de execução para esta máquina de estados teve início a 18 de maio de 2026. Os primeiros testes revelaram erros de sincronização entre os índices `iIndex` e `jIndex`, corrigidos numa segunda ronda no início de junho; uma versão posterior, designada internamente por FSM 2.0 e concluída a 2 de junho de 2026, acrescentou o suporte a movimento nos dois sentidos e a pausas configuráveis no início e no fim de cada trajetória.

Interação com as estruturas de dados. A cada chamada, a FSM lê da `MotionStorage` associada (via `inst.def`): o número de pontos `nPoints`, o período total `period`, o intervalo por segmento `dtMs`, o índice inicial `start_Index`, o flag `bidirectional` e as durações de pausa `Dt_pauseMs_Ini/Dt_pauseMs_End`. Escreve na `MotionInstance`: os índices `iIndex` e `jIndex`, os cronómetros `elapsed_1`, `elapsed_2` e `segmentStartMs`, a direção `Direction` e a flag `Executing`.

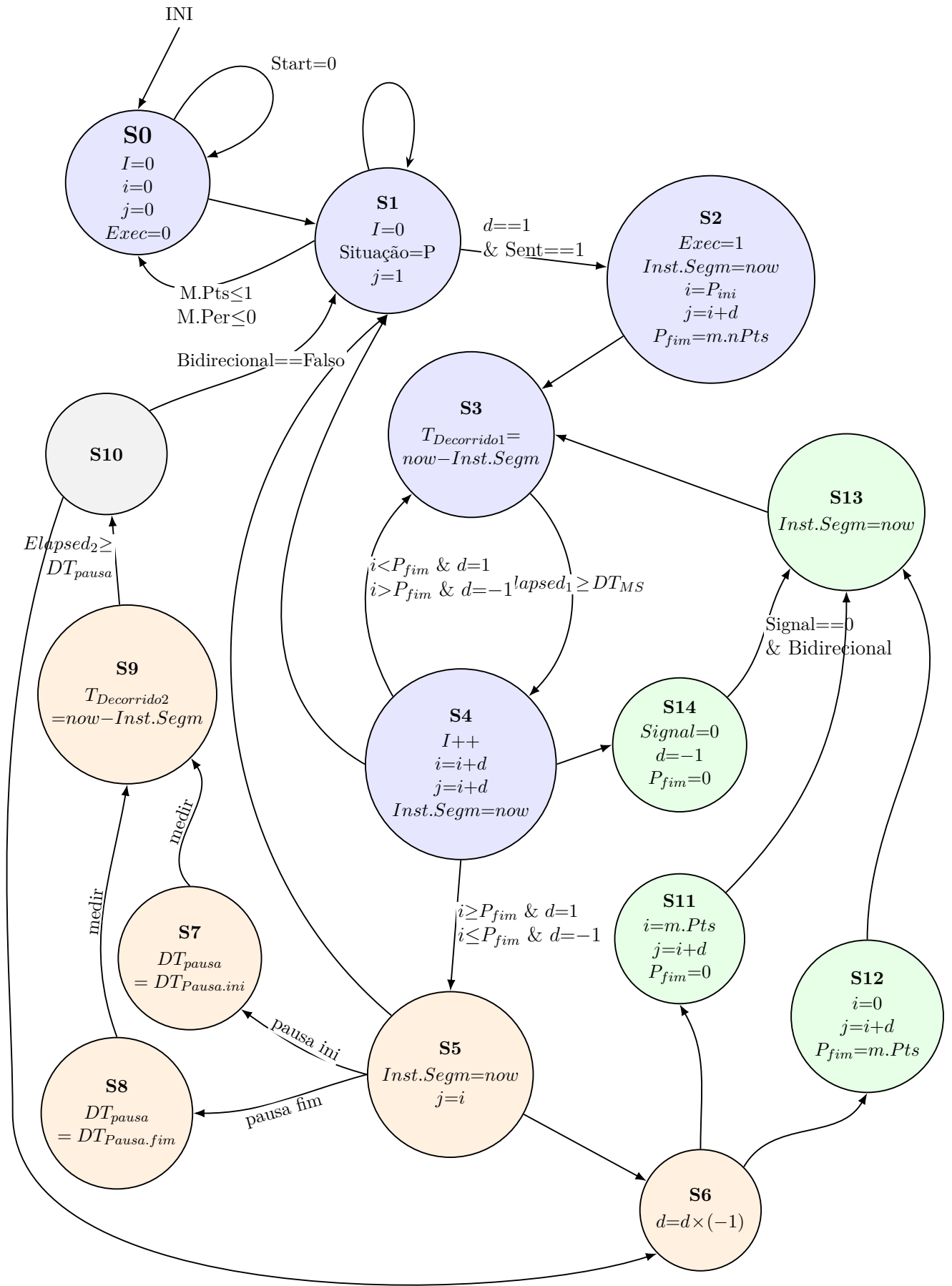


Figura 4.7: Esquema da FSM_Motion_Update.

Fases de funcionamento.

- **Arranque (S0→S1→S2):** No estado S0 a instância está inativa, com índices e `Executing` zerados. Quando `start=1` e a instância está marcada como ativa, a FSM transita para S1, que valida os parâmetros da trajetória (pelo menos dois pontos e período positivo) e inicializa `Direction=1`. Em S2 regista `segmentStartMs=nowMs`, define `iIndex=start_Index` e `jIndex=iIndex+d`, e transita imediatamente para S3.
- **Ciclo de segmentos (S3↔S4):** S3 mede o tempo decorrido (`elapsed_1 = nowMs - segmentStartMs`) e aguarda que este iguale `dtMs`. Quando atingido, S4 avança `iIndex` na direção `d` (+1 ou -1), atualiza `jIndex=iIndex+d`, reinicia o cronómetro e retorna a S3. Este ciclo repete-se até `iIndex` atingir `End_Point`.
- **Gestão do extremo e pausas (S5→S7/S8→S9→S10):** Ao atingir o fim da passagem, S5 decide o caminho: se existir pausa configurada, S7 ou S8 carregam a duração correspondente (inicial ou final), S9 aguarda até ao fim da pausa, e S10 decide se recomeça o ciclo bidirecional (→S6) ou reinicializa do início (→S1).
- **Inversão de direção (S6→S11/S12→S13):** S6 inverte `Direction` (multiplica por -1). Consoante o novo sentido, S11 (passagem descendente) ou S12 (passagem ascendente) reinicializa os índices para o extremo correto. S13 regista o novo `segmentStartMs` e entra de imediato em S3.
- **Interrupção de tosse (S4→S14→S13):** Se o sinal global `Tosse_signal` for ativado durante o avanço (S4), a FSM entra em S14, que força a direção de regresso (`Direction=-1`, `End_Point=0`) e encadeia com S13 para iniciar a descida de emergência.

Saída para a função de posição instantânea. Ao terminar cada chamada, a `MotionInstance` contém: `iIndex` (índice do ponto de partida do segmento atual), `jIndex` (índice do ponto de chegada) e `elapsed_1` (tempo decorrido desde o início do segmento). A função `Intermed_position()` lê estes três campos, acede ao `MotionStorage` para obter as coordenadas `X[i]`, `X[j]`, `Y[i]`, `Y[j]`, `Z[i]`, `Z[j]` e o intervalo `dtMs`, e calcula a posição interpolada entre os dois pontos. O resultado é um triploto (X, Y, Z) em coordenadas relativas ao ponto de referência `Index0`, que segue depois para a função de fusão e cinemática inversa.

A posição instantânea é obtida por interpolação linear entre dois pontos consecutivos da trajetória:

$$P(\alpha) = P_i + (P_j - P_i) \alpha$$

Definindo a razão temporal bruta $\alpha_0 = t_{dec}/DT_{MS}$, onde t_{dec} é o valor de `elapsed_1` e

DT_{MS} é $\mathbf{dtMs}$, o coeficiente de interpolação saturado é:

$$\alpha = \begin{cases} 0 & \text{se } \alpha_0 < 0 \\ \alpha_0 & \text{se } 0 \leq \alpha_0 \leq 1 \\ 1 & \text{se } \alpha_0 > 1 \end{cases} \quad \alpha_0 = \frac{t_{dec}}{DT_{MS}}$$

4.6 Cinemática inversa do manipulador delta

A cinemática inversa converte a posição desejada da plataforma móvel (X, Y, Z) nos três ângulos de servo θ_1 , θ_2 e θ_3 . A figura 4.8 mostra a representação lateral usada como referência para as variáveis do cálculo: L_1 representa o braço acionado pelo servo, L_2 o braço passivo, L_3 o afastamento horizontal até à articulação da plataforma móvel, ϕ e ω são ângulos intermédios e θ é o ângulo final aplicado ao servo.

Nas primeiras experiências com este problema, antes de se fixar a abordagem final, foram consultados dois recursos que representam uma família de solução diferente: a biblioteca *Arduino Delta-Kinematics-Library* [56] e o tutorial de Nikanorov [57]. Ambos resolvem a cinemática inversa por interseção geométrica entre uma circunferência, centrada no eixo do servo e com raio igual ao braço ativo, e uma esfera centrada na plataforma com raio igual ao braço passivo, parametrizando o problema com um raio de base (f) e um raio de plataforma (e) em vez dos comprimentos L_1 , L_2 e L_3 usados neste projeto, e obtendo o ângulo final diretamente por atan2 da interseção, sem uma etapa explícita de lei dos cossenos. Esta abordagem foi testada nos primeiros esboços de código, mas revelou-se menos direta de adaptar à disposição mecânica dos servos e ao referencial lateral usados no desenho dos manipuladores deste projeto, pelo que se optou por explorar, em alternativa, a decomposição por lei dos cossenos seguida de atan2 — matematicamente equivalente, mas mais fácil de mapear diretamente às variáveis `SERVO_OFFSET_X/SERVO_OFFSET_Z` do desenho mecânico.

A primeira versão funcional deste cálculo foi implementada no final de abril de 2026, com base num repositório de código aberto de um manipulador delta impresso em 3D e controlado por Arduino [58], aplicada isoladamente a um único braço do manipulador e validada através de um movimento linear simples. A meio de maio, a função foi generalizada para servir os dois manipuladores delta a partir da mesma base de código, eliminando a duplicação que existia entre os dois braços e permitindo a sincronização dos respetivos movimentos. Esta abordagem geométrica — decomposição por braço com simetria de 120° , resolvendo cada perna pela lei dos cossenos seguida de atan2 — segue a família de soluções estabelecida por Zsombor-Murray para a cinemática do robô delta de Clavel [59], e mantém-se consistente com formulações mais recentes do mesmo problema [60].

O firmware aplica o mesmo cálculo aos três braços. A diferença entre eles está

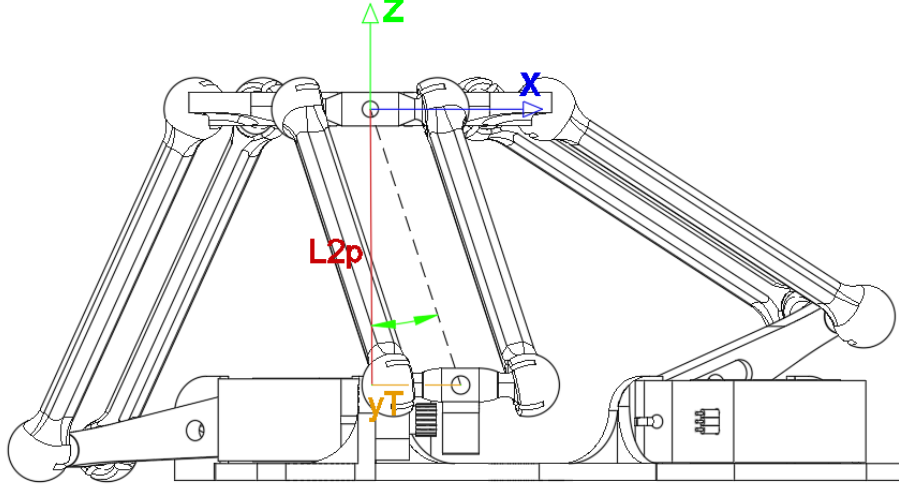


Figura 4.8: Representação lateral das variáveis usadas na cinemática inversa do manipulador delta.

apenas na rotação do referencial no plano XY . Para cada manipulador existe um offset global α_m , definido na estrutura `Manipulador_Config`, e para cada servo existe uma rotação adicional β_k :

$$\alpha_m = \begin{cases} -15^\circ, & \text{Manipulador Delta 1} \\ -45^\circ, & \text{Manipulador Delta 2} \end{cases} \quad \beta_k \in \{0^\circ, 120^\circ, 240^\circ\}$$

$$\varphi_k = \alpha_m + \beta_k \quad R_z(\varphi_k) = \begin{bmatrix} \cos \varphi_k & -\sin \varphi_k & 0 \\ \sin \varphi_k & \cos \varphi_k & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Assim, o ponto pedido é primeiro expresso no plano de trabalho do servo k :

$$\begin{bmatrix} x'_k \\ y'_k \\ z'_k \end{bmatrix} = R_z(\varphi_k) \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ z_{offset} \end{bmatrix} \implies \begin{cases} x'_k = X \cos \varphi_k - Y \sin \varphi_k \\ y'_k = X \sin \varphi_k + Y \cos \varphi_k \\ z'_k = Z - z_{offset} \end{cases}$$

No código, $z_{offset} = 0$ mm, mas a variável foi mantida para permitir calibração posterior.

Depois da rotação, o problema fica reduzido ao plano lateral do servo. A extremidade associada à plataforma móvel é deslocada pelo comprimento L_3 :

$$x_{arm,k} = x'_k + L_3$$

Como o braço passivo L_2 pode sair ligeiramente do plano, o código calcula a sua projeção nesse plano. Esta projeção só é válida se a junta esférica não ultrapassar o limite angular

admitido:

$$L_{2p,k} = \sqrt{L_2^2 - y_k'^2} \quad \gamma_k = \arcsin\left(\frac{y_k'}{L_2}\right) \quad |\gamma_k| < 0,5934 \text{ rad} \approx 34^\circ$$

Este limite aparece no firmware como proteção para evitar que o ângulo entre a junta esférica e o braço passivo saia da zona mecânica admissível.

De seguida calcula-se a distância entre o eixo do servo e a articulação projetada. Esta distância é designada no código por **ext**:

$$d_k = \sqrt{z_k'^2 + (d_s - x_{arm,k})^2} \quad d_s = 32 \text{ mm}$$

Antes de calcular o ângulo, o firmware confirma se os comprimentos L_1 , $L_{2p,k}$ e d_k conseguem formar um triângulo:

$$L_{2p,k} - L_1 < d_k < L_1 + L_{2p,k}$$

Se esta condição falhar, a posição pretendida fica fora do alcance geométrico daquele braço e a função de cinemática inversa devolve erro.

Com a geometria validada, o ângulo interno ϕ_k é obtido pela lei dos cossenos, enquanto ω_k corresponde à inclinação da reta entre o pivô do servo e a articulação projetada:

$$\phi_k = \arccos\left(\frac{L_1^2 + d_k^2 - L_{2p,k}^2}{2L_1d_k}\right) \quad \omega_k = \text{atan2}(z_k', d_s - x_{arm,k})$$

O ângulo enviado ao servo é então:

$$\theta_k = \phi_k + \omega_k \quad 45^\circ \leq \theta_k \leq 225^\circ$$

O intervalo de 45° a 225° é o limite angular definido no firmware por **SERVO_ANGLE_MIN** e **SERVO_ANGLE_MAX**, pelo que, se algum dos três ângulos calculados ficar fora deste intervalo, a posição é rejeitada e os servos não são atualizados.

Por fim, cada θ_k é convertido para largura de pulso PWM por interpolação linear:

$$PWM_k = PWM_{k,min} + \frac{\theta_k - \theta_{min}}{\theta_{max} - \theta_{min}} (PWM_{k,max} - PWM_{k,min})$$

Considerando os valores de referência $PWM_{min} = 500 \mu s$ e $PWM_{max} = 2500 \mu s$, a relação é crescente no Manipulador Delta 1 e invertida no Manipulador Delta 2, devido à montagem mecânica oposta dos servos. Esta inversão é feita no arranque do firmware, quando os limites **thetaMinPulse** e **thetaMaxPulse** do segundo manipulador são trocados.

Os limites de PWM usados no código não foram assumidos apenas a partir de valores teóricos do servo. Estes valores foram calibrados experimentalmente, começando

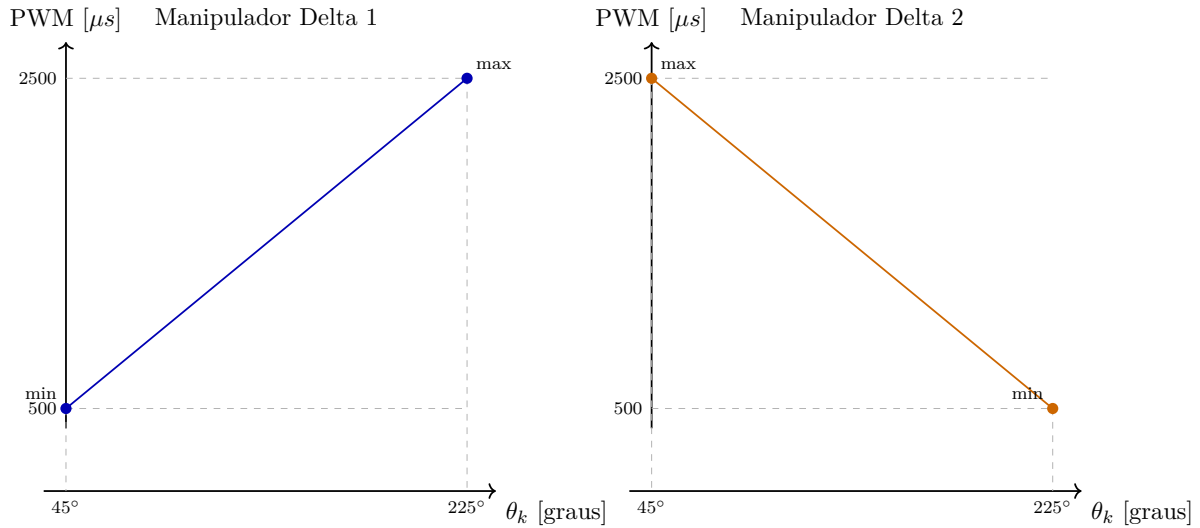


Figura 4.9: Relação linear de referência entre o ângulo calculado pela cinemática inversa e o pulso PWM aplicado aos servos nos dois manipuladores.

pelo alinhamento da patilha de cada servo com a base onde estava acoplado. O objetivo deste alinhamento era garantir que as posições físicas de 0° e 180° ficavam paralelas à superfície da base. Depois desse alinhamento mecânico, era feita a transição manual da referência física para o intervalo usado pela cinemática: no Manipulador Delta 1, de 0° para 45°; no Manipulador Delta 2, devido à orientação oposta dos servos, de 180° para 45°. Por fim, foi necessário repetir ensaios de calibração para confirmar que os impulsos PWM correspondiam aos limites de 45° e 225°, admitindo uma tolerância aproximada de $\pm 5^\circ$.

$$\text{Manipulador Delta 1 - Calibrado} - \left\{ \begin{array}{ll} \text{Mínimo} & \text{Máximo} \\ \text{Servo } \theta_1 : & 520 \mu s \quad 2520 \mu s \\ \text{Servo } \theta_2 : & 550 \mu s \quad 2550 \mu s \\ \text{Servo } \theta_3 : & 560 \mu s \quad 2560 \mu s \end{array} \right.$$

$$\text{Manipulador Delta 2 - Calibrado} - \left\{ \begin{array}{ll} \text{Mínimo} & \text{Máximo} \\ \text{Servo } \theta_1 : & 460 \mu s \quad 2460 \mu s \\ \text{Servo } \theta_2 : & 470 \mu s \quad 2470 \mu s \\ \text{Servo } \theta_3 : & 420 \mu s \quad 2420 \mu s \end{array} \right.$$

No arranque, o firmware mantém estes intervalos físicos para associar cada servo ao seu canal PWM, mas inverte a correspondência entre `thetaMinPulse` e `thetaMaxPulse` no Manipulador Delta 2. Assim, a cinemática inversa continua a gerar os mesmos três ângulos lógicos, enquanto a conversão final para PWM respeita a montagem, o sentido de

rotação e a calibração individual de cada servo.

4.7 Comunicação série

4.7.1 Comunicação série no firmware (ESP32)

A comunicação série é organizada por duas máquinas de estados principais: a primeira é a `FSM_Serial_reader()`, responsável apenas pela recepção dos dados que chegam pela porta série, e a segunda é a `FSM_Serial_Command()`, que interpreta esses dados e altera o comportamento do sistema, sendo esta separação usada precisamente para evitar que a leitura da porta série ficasse misturada com a lógica de controlo dos movimentos.

A primeira versão desta separação em duas máquinas de estados ficou operacional a 8 de junho de 2026. Nas semanas seguintes, à medida que a interface gráfica do Raspberry Pi ia sendo desenvolvida, os comandos disponíveis foram sendo atualizados e ajustados para corresponder às ações que a HMI passou a expor ao utilizador.

A `FSM_Serial_reader()` funciona como uma camada de aquisição. Quando o sistema está em modo simples, lê um único carácter e coloca esse valor em `CH_Command`. Quando é ativado o modo de comando longo, através do comando `W`, passa a preencher o vetor `ST_Bus` até receber o fim da mensagem. Só depois de a mensagem estar completa é que ativa a flag de linha pronta. Assim, a máquina de comandos nunca precisa de ler diretamente da porta série; apenas consome comandos já preparados.

A `FSM_Serial_Command()` funciona como a camada de decisão. Esta máquina verifica se existe um comando disponível, interpreta o carácter ou a cadeia recebida, atualiza flags como `start_comand`, `Safe_comand` e `sinai_tossir`, seleciona que movimentos ficam ativos e, nos comandos longos, altera parâmetros das estruturas `MotionStorage`. Usar duas máquinas de estados torna o comportamento mais previsível: a recepção pode continuar a ser tratada passo a passo, enquanto a interpretação dos comandos fica concentrada numa rotina própria.

Os principais comandos simples disponíveis são:

Tabela 4.4: Comandos simples disponíveis na comunicação série.

Comando	Função
S	Inicia a execução do ciclo de movimentos.
P	Para a execução e calcula a duração do ensaio.
L	Entra no menu de leitura, teste e depuração.
M	Entra no menu de seleção do modo fisiológico.
W	Ativa o modo de comandos com vários caracteres.

No menu L, o segundo carácter define a ação a executar:

No menu M, o segundo carácter escolhe que movimentos ficam ativos:

Tabela 4.5: Subcomandos do menu de leitura, teste e depuração.

Comando	Função
L1	Ativa o comando de segurança definido por <code>Safe_comand = 1</code> .
L2	Desativa esse comando de segurança, colocando <code>Safe_comand = 0</code> .
L3	Ativa o sinal de tosse, através de <code>sinal_tossir = 1</code> .
Lr	Imprime os dados de respiração, <code>Resp_move</code> e <code>Resp_inst</code> .
Lb	Imprime os dados de batimento, <code>Bat_move</code> e <code>Bat_inst</code> .
Lt	Reservado para leitura dos dados de tosse.
La	Imprime em conjunto os dados principais de respiração e batimento.

Tabela 4.6: Subcomandos de seleção do modo fisiológico.

Comando	Modo selecionado
Ma	Ativa respiração, batimento e tosse.
Mh	Ativa o modo humano, com respiração e batimento.
Mr	Ativa apenas a respiração.
Mb	Ativa apenas o batimento.
Ms	Desativa os movimentos, deixando o sistema em modo estático.

Os comandos longos começam por W e permitem enviar uma cadeia de caracteres para alterar parâmetros numéricos. No estado atual do firmware, a estrutura implementada permite selecionar o movimento de destino com R, B ou T, correspondentes a respiração, batimento e tosse, e depois alterar o período com o campo `T[valor]`. Por exemplo, uma mensagem do tipo `WdRT[4.0]` é interpretada como uma alteração do período da respiração para 4,0 s, de acordo com o formato que a máquina de estados procura no vetor `ST_Command`. Os campos para pausa e alteração de pontos de movimento aparecem previstos na máquina, mas ainda não estão totalmente desenvolvidos no código.

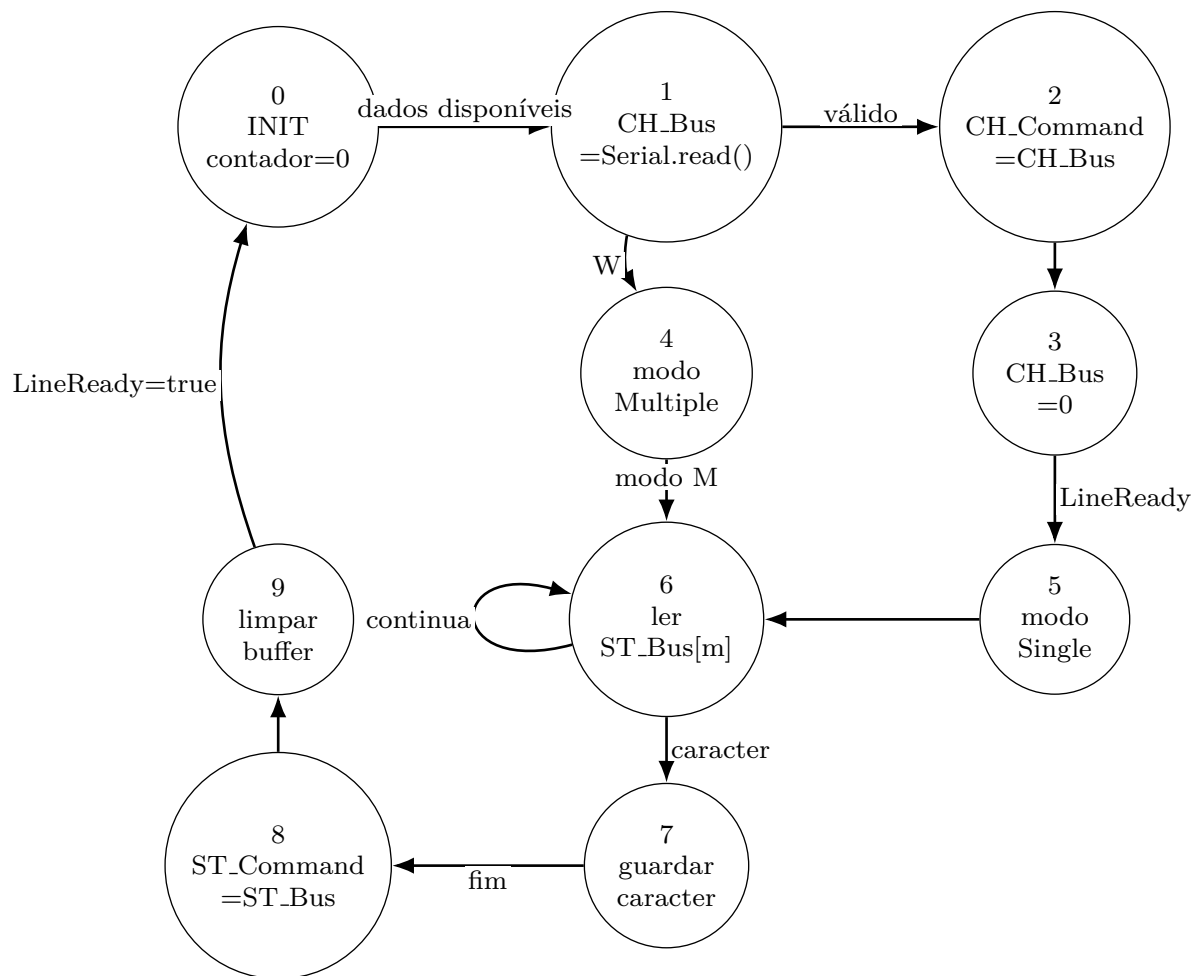


Figura 4.10: `FSM.Serial_Read` responsável pela leitura e validação dos dados recebidos por comunicação série.

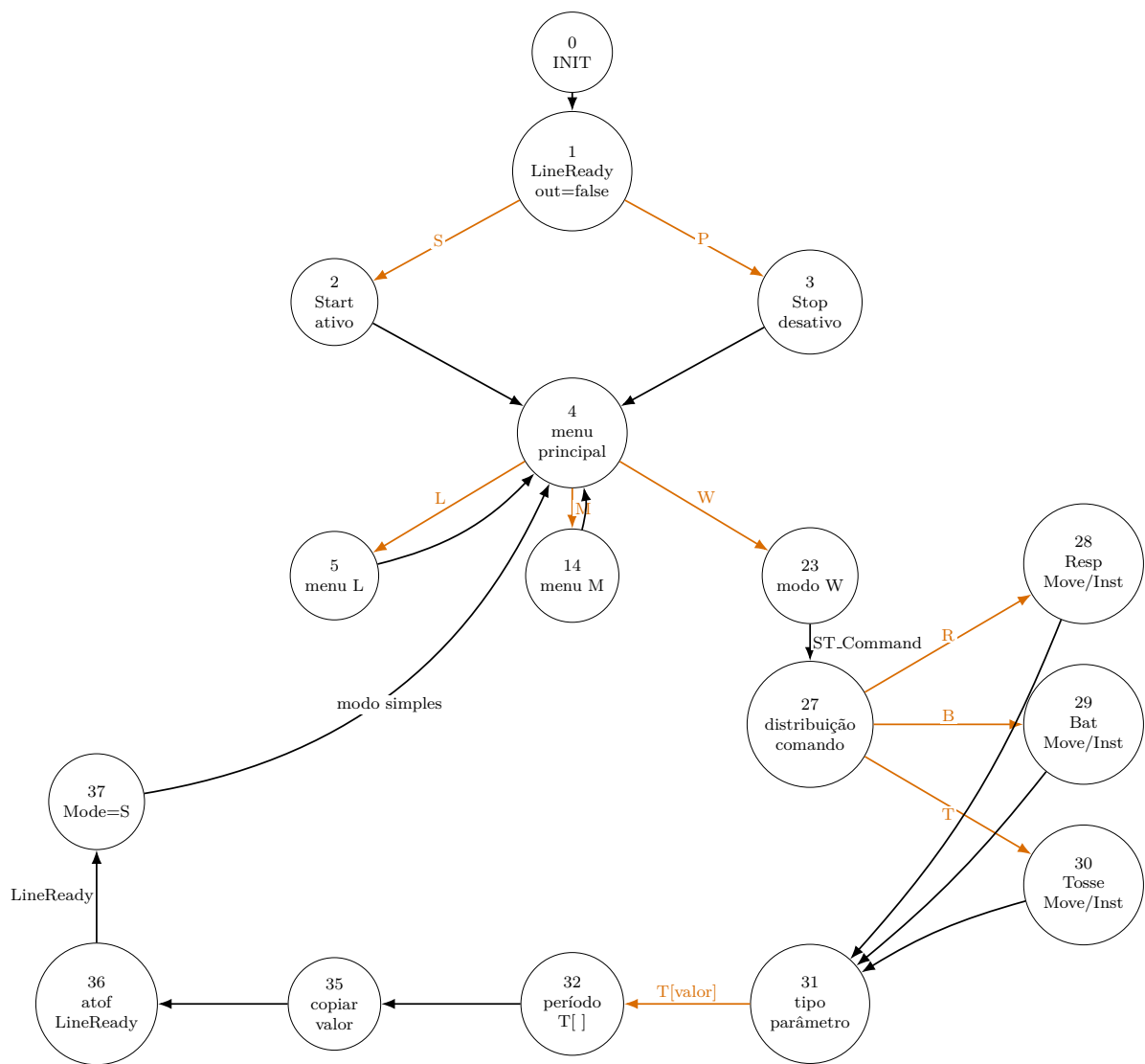


Figura 4.11: FSM_Serial_Command responsável pela interpretação dos comandos recebidos por comunicação série.

4.7.2 Comunicação série no Python (Raspberry Pi)

(conteúdo a enviar, ficheiros separados de comunicação série do lado do Raspberry Pi/Python)

4.8 Interface HMI no Raspberry Pi

A interface homem-máquina foi desenvolvida para transformar os comandos de baixo nível do firmware numa utilização mais simples e natural, já que, sem esta camada, o utilizador teria de memorizar comandos como **S**, **P**, **Mr**, **Mb**, **Mh** ou **Ma**, escrevendo-os diretamente no monitor série; com a HMI, essas ações passam antes a estar disponíveis através de botões, menus e páginas de configuração, reduzindo a probabilidade de erro e tornando o modelo mais adequado para demonstração e treino.

A primeira versão funcional desta interface, identificada internamente como `Interface_V1`, ficou disponível a 19 de junho de 2026. Nas semanas seguintes foi sujeita a várias rondas de revisão: primeiro ao nível da comunicação série e da organização do repositório (19–26 de junho), depois com modificações à disposição gráfica em modo cliente e DEV (26 de junho) e, já na reta final do projeto, com a correção de erros na troca de dados entre a interface e o firmware e melhorias adicionais à página de monitorização (1–3 de julho de 2026).

Na versão para Raspberry Pi, o ficheiro principal da interface é `RB.PI4B.Main.PI.py`. A aplicação usa Flask para servir as páginas web e Flask-SocketIO para enviar atualizações em tempo real aos browsers ligados. Esta arquitetura permite ter uma interface local no monitor do Raspberry Pi e, ao mesmo tempo, uma interface móvel acessível por telemóvel ou tablet na mesma rede. Para facilitar esse acesso, a aplicação gera um endereço local e um código QR que aponta para a rota `/mobile`.

A comunicação com o ESP32 é feita por uma porta série a 115200 baud. A interface tenta detetar automaticamente a porta mais provável no Linux, dando prioridade a dispositivos em `/dev/serial/by-id`, `/dev/ttyUSB*` e `/dev/ttyACM*`. A leitura e a escrita série são executadas em threads separadas: uma thread lê continuamente as mensagens recebidas do ESP32 e atualiza o histórico, enquanto outra consome uma fila de transmissão e envia os comandos pedidos pela interface. Esta separação evita que a interface fique bloqueada durante leituras ou esperas da porta série.

O estado da aplicação é guardado numa estrutura partilhada que contém, entre outros campos, o modo de desenvolvimento, o histórico da comunicação série, o estado de arranque/paragem e o modo fisiológico ativo. Sempre que algum destes valores muda, a função de atualização emite um evento SocketIO para todos os browsers ligados. Assim, se o utilizador abrir simultaneamente a interface no Raspberry Pi e no telemóvel, ambos os ecrãs podem refletir o mesmo estado de funcionamento.

As ações principais da HMI são:

- **Start/Stop:** os botões de arranque e paragem enviam, respetivamente, os comandos `s` e `p` para o ESP32, alterando também o estado visual da interface;
- **Seleção de modo:** o menu de modos converte escolhas como estático, respiração, coração, humano e completo nos comandos série `Ms`, `Mr`, `Mb`, `Mh` e `Ma`;
- **Ligação série:** a interface permite ligar, desligar e tentar reconectar a comunicação com o ESP32, mostrando o estado como desligado, a ligar, ligado ou erro;
- **Monitor série:** em modo de desenvolvimento, o utilizador pode observar as mensagens recebidas do ESP32 e enviar comandos manuais, o que mantém uma via de depuração direta;
- **Modo cliente e modo DEV:** o modo cliente apresenta apenas as ações necessárias à utilização normal, enquanto o modo DEV, protegido por palavra-passe, revela ferramentas de depuração, envio manual de comandos e encerramento completo da aplicação;
- **Configurações de movimentos:** a interface permite editar parâmetros associados à respiração, batimento e tosse, guardar configurações em ficheiro JSON e gerar pontos de movimento a partir do módulo `calcula_movimentos.py`.

A página de configurações de movimentos usa parâmetros geométricos para gerar curvas de referência. A curva elíptica é usada para representar o batimento cardíaco, a curva parabólica no plano ZX é usada para movimentos de respiração/tosse e a circunferência no plano XY é usada para vibração da tosse. O módulo de cálculo devolve gráficos 2D e 3D em formato de imagem codificada em base64, permitindo que os resultados sejam apresentados diretamente na interface web sem abrir janelas externas. Os pontos calculados são guardados nas configurações e podem servir de base para a atualização posterior dos movimentos no firmware.

O desenvolvimento da HMI foi feito depois de validadas duas camadas de suporte. A primeira foi a comunicação série elementar, testada com scripts Python que liam mensagens do ESP32, enviavam comandos escritos no terminal e confirmavam o efeito dos sinais DTR/RTS no reset da placa. A segunda foi o cálculo isolado das trajetórias, testando a geração de pontos e gráficos antes da sua integração na aplicação Flask. Esta ordem de desenvolvimento reduziu a incerteza: quando a interface foi construída, já estava confirmado que o canal série e a lógica de geração de movimentos funcionavam de forma independente.

4.9 Preparação e sincronização da tosse

Esta funcionalidade foi implementada numa fase avançada do desenvolvimento, a partir de 24 de junho de 2026, depois de os movimentos de respiração e batimento cardíaco já estarem validados de forma independente — o que permitiu tratar a tosse como uma alteração temporária a um comportamento já existente, em vez de partir do zero.

A função de preparação da tosse é responsável por transformar um pedido de tosse num evento fisiológico sobreposto à respiração. Quando o sinal de tosse é ativado, a rotina guarda os parâmetros normais da respiração, reduz temporariamente o período, ajusta as pausas e aumenta a amplitude em Z . Em paralelo, ativa a instância de vibração ou perturbação da tosse.

Quando o ciclo respiratório atinge o estado apropriado para terminar o evento, a rotina repõe o período original, as pausas e o multiplicador de amplitude. Esta estratégia permite que a tosse não seja apenas uma trajetória independente, mas uma alteração temporária do comportamento respiratório, mais coerente com o fenómeno fisiológico que se pretende simular.

4.10 Depuração e validação interna

A validação interna do firmware foi feita em paralelo com o desenvolvimento das máquinas de estados. Como o sistema junta comunicação série, geração de trajetórias, cinemática inversa e acionamento de seis servos, foi necessário criar pontos de observação que permitissem perceber em que parte da cadeia surgia um erro. Para isso foram usadas mensagens enviadas pela porta série e funções auxiliares de impressão, principalmente `debugPrintMove()` e `debugPrintInstance()`.

A função `debugPrintMove()` permite confirmar os dados guardados numa estrutura `MotionStorage`: número de pontos, período, intervalo temporal entre pontos e coordenadas X , Y e Z de cada posição. Esta verificação foi importante para confirmar se os movimentos de respiração, batimento e tosse estavam a ser carregados com os valores esperados antes de serem usados pela interpolação. A função `debugPrintInstance()` foi usada para observar o estado de execução de cada movimento, incluindo se a instância estava ativa, qual o índice atual e qual a estrutura de movimento associada.

Foram também mantidos testes isolados para componentes específicos. Os testes ao LED RGB permitiam confirmar rapidamente o estado dos comandos recebidos, uma vez que diferentes comandos alteravam a cor do LED. Os testes aos servos, por sua vez, ajudavam a verificar se os canais PWM estavam corretamente associados, se os limites calibrados eram respeitados e se os servos respondiam no sentido esperado. Esta validação por partes foi essencial para distinguir erros de software, como uma transição incorreta numa máquina de estados, de problemas de montagem ou calibração.

Outro aspeto observado durante os ensaios foi o comportamento da porta série no ESP32-S3. Ao abrir o monitor série ou alguns scripts Python, os sinais DTR/RTS podem provocar o reset da placa. Quando isso acontece, o `setup()` volta a ser executado, os servos são novamente associados aos canais PWM e o sistema regressa ao estado inicial. Por este motivo, durante os testes é necessário ter em conta que a abertura da porta série pode alterar temporariamente o estado do sistema, mesmo sem ter sido enviado um comando de movimento.

Assim, a depuração não foi usada apenas para imprimir valores no terminal, mas como uma ferramenta de validação do fluxo completo: receção do comando, interpretação pela máquina de estados, atualização das estruturas de movimento, cálculo da posição intermédia, aplicação da cinemática inversa e envio final dos impulsos PWM para os servos.

Esta prática acompanhou praticamente todo o desenvolvimento do firmware, tendo-se tornado particularmente intensa na fase final do projeto, entre o final de junho e o início de julho de 2026, quando a integração entre o firmware e a interface gráfica do Raspberry Pi expôs a maioria dos erros de sincronização e de formatação dos comandos série.

Capítulo 5

Testes e Análise de Resultados

Nota sobre o estado atual do capítulo

Este capítulo encontra-se em desenvolvimento por falta de informação experimental completa, uma vez que, nesta fase do relatório, ainda não existem resultados quantitativos suficientes para apresentar uma análise final de desempenho. Assim, o capítulo descreve o plano de testes, os critérios de aceitação e a informação que deve ser recolhida nas próximas iterações do projeto.

5.1 Plano de testes

O plano de testes foi definido para validar separadamente o modelo mecânico, a eletrónica, o firmware e a integração completa, seguindo uma estratégia que consiste em testar primeiro cada subsistema de forma isolada e, depois, combinar progressivamente respiração, batimento cardíaco e tosse.

Para cada ensaio devem ser registados: configuração usada, período, amplitude, número de pontos da trajetória, modo de operação, comportamento observado, eventuais falhas, temperatura dos atuadores, estabilidade da alimentação e repetibilidade do movimento. A comparação deve ser feita com os valores-alvo definidos para os movimentos representados no protótipo.

5.2 Testes do modelo físico

Os primeiros testes devem confirmar se o modelo impresso mantém a geometria pretendida e se permite a passagem adequada do broncoscópio, observando-se ainda deformações, zonas frágeis, arestas, dificuldade de limpeza e resistência ao manuseamento. No caso de materiais flexíveis, deve também ser avaliada a relação entre flexibilidade e estabilidade dimensional.

O teste inicial em PLA serve como prova de conceito da geometria e da montagem, ainda que, por ser um material rígido, não represente a solução final para realismo mecânico. Por isso, ensaios posteriores com TPU, TPE ou outros filamentos flexíveis devem comparar facilidade de impressão, acabamento, deformação elástica e resistência à repetição dos movimentos.

5.3 Testes dos atuadores e da estrutura delta

Os servos devem ser testados antes da integração com a árvore brônquica, incluindo verificação de resposta angular, limites de curso, sentido de rotação, ruído, aquecimento e repetibilidade. A seguir, deve ser testado o manipulador delta sem carga, confirmando se a cinemática inversa produz movimentos coerentes nos eixos X , Y e Z .

Depois da montagem com o modelo, os ensaios devem verificar se os movimentos esperados são obtidos sem folgas excessivas nem bloqueios. As amplitudes de referência são 5 a 10 mm para respiração ao nível da traqueia, 1 a 3 mm para batimento cardíaco e valores superiores, de curta duração, para tosse. Se o deslocamento real for inferior ao pretendido, deve ser ajustada a amplitude dos pontos da trajetória ou a geometria de ligação.

5.4 Testes de alimentação

A alimentação deve ser validada em três condições, repouso, movimento nominal e situação de maior esforço, considerando os consumos estimados para os servos já apresentados no section 3.4, bem como correntes superiores expectáveis em bloqueio. Assim, a fonte dos servos deve ser dimensionada para picos e não apenas para consumo médio.

O Raspberry Pi 4B e o ESP32-S3 devem manter alimentação estável durante a comunicação e durante o acionamento dos motores. A utilização de UPS/HAT deve ser testada abrindo e fechando a alimentação principal, confirmando se o sistema tem tempo para guardar estado, alertar ou encerrar de forma controlada.

5.5 Testes de software

Os testes de software devem começar pela inicialização das estruturas `MotionStorage` e `MotionInstance`, cujas funções de depuração devem confirmar o número de pontos, período, pausas, índices e ponteiros. Em seguida, a FSM de movimento deve ser testada com trajetórias simples, verificando se os estados evoluem de forma esperada e se a interpolação entre pontos é contínua.

A FSM de comandos deve ser testada com entradas série conhecidas, de forma a que cada comando produza apenas a alteração prevista: ativar respiração, ativar bati-

mento, ativar modo combinado, disparar tosse ou alterar parâmetros. Já os comandos com parâmetros numéricos devem ser testados com valores válidos, valores limite e entradas incompletas.

5.6 Testes de comunicação série

Durante o desenvolvimento foi identificado que a abertura da porta série por Python ou pelo monitor série pode provocar reset do ESP32, pelo que este efeito deve ser controlado durante os ensaios. Um teste simples consiste em imprimir um contador no `setup()` e observar se este é reiniciado quando a porta é aberta, o que permite distinguir falhas de firmware de resets causados pela interface USB/serial.

No lado Python, a leitura deve remover caracteres de fim de linha com `strip()` e evitar imprimir mensagens vazias. A comunicação deve ser validada em dois sentidos: mensagens enviadas pelo ESP32 para monitorização e comandos enviados pelo computador ou Raspberry Pi para selecionar modos.

5.7 Testes de integração dos movimentos

A integração deve ser feita por etapas: primeiro executa-se apenas respiração, verificando periodicidade, amplitude e suavidade; depois adiciona-se o batimento cardíaco, que deve aparecer como pequena oscilação sobreposta; por fim, testa-se a tosse, que deve alterar temporariamente o comportamento da respiração e ativar vibração ou perturbação curta.

O modo combinado é o mais importante para avaliar o realismo, pois aproxima o comportamento de um ambiente broncoscópico dinâmico. Neste modo, devem ser observados conflitos entre movimentos, saturação dos servos, perda de suavidade e eventuais posições impossíveis da cinemática inversa.

5.8 Critérios de aceitação

Para esta fase do projeto, os critérios de aceitação podem ser definidos de forma funcional:

- o sistema arranca sem movimentos inesperados perigosos;
- os servos respondem dentro dos limites definidos;
- a respiração apresenta movimento cíclico, suave e repetível;
- o batimento acrescenta uma oscilação de pequena amplitude;
- a tosse produz um evento curto e abrupto, recuperando depois para o estado normal;

- os comandos série selecionam os modos sem bloquear o firmware;
- a alimentação mantém tensão estável durante os movimentos.

5.9 Informação experimental em falta

Para transformar este plano numa análise final de resultados, falta recolher dados quantitativos. Em particular, devem ser medidos deslocamentos reais nos eixos X , Y e Z , frequência efetiva dos movimentos, erro entre posição desejada e posição observada, consumo elétrico, aquecimento dos servos, estabilidade da fonte e tempo de resposta aos comandos série.

Também falta documentar a avaliação visual do movimento com o broncoscópio introduzido no modelo. Esta etapa é importante porque o realismo percebido não depende apenas da amplitude mecânica, mas também da forma como o movimento aparece na imagem broncoscópica.

Foi realizada uma sessão preliminar de observação com um profissional de saúde, em que o broncoscópio foi introduzido no modelo impresso em 3D enquanto o sistema estava ativo (figura 5.1). Esta sessão permitiu obter uma primeira avaliação qualitativa da disposição dos ramos, do espaço disponível para a progressão da sonda e da percepção de movimento durante o funcionamento do manipulador. As observações desta sessão ajudaram a identificar aspetos a melhorar na geometria do modelo e no comportamento do sistema, mas os dados quantitativos ainda não foram recolhidos de forma sistemática.



Figura 5.1: Sessão de observação preliminar com profissional de saúde: broncoscópio introduzido no modelo de árvore brônquica impresso em 3D enquanto o sistema de controle estava ativo, junho de 2026.

Capítulo 6

Conclusão

6.1 Síntese do trabalho

Este projeto tem como objetivo desenvolver um modelo articulado de uma árvore brônquica para treino de videobroncofibroscopia, cuja solução proposta combina impressão 3D, atuação mecânica, controle por ESP32, comunicação com Raspberry Pi 4B, interface HMI e perfis de movimento inspirados em fenômenos fisiológicos.

O desenvolvimento realizado permitiu consolidar os requisitos do sistema, segundo os quais o modelo deve ser acessível, modular, fácil de montar e capaz de reproduzir movimentos relevantes durante uma broncoscopia flexível. Para esta versão, foram priorizados três fenômenos, respiração, batimento cardíaco e tosse, tendo outros efeitos, como deformação localizada do lúmen, estenose e resistência ao contacto com o broncoscópio, sido identificados como importantes mas reservados para trabalho futuro devido à maior complexidade mecânica.

6.2 Principais contributos

Os principais contributos do trabalho são:

- definição de requisitos para um simulador dinâmico de árvore brônquica;
- identificação dos movimentos fisiológicos mais relevantes para uma primeira versão funcional;
- escolha de uma arquitetura mecânica baseada em manipuladores delta;
- definição da arquitetura eletrónica com Raspberry Pi 4B, ESP32-S3, servos e sistema de alimentação;
- desenvolvimento de uma arquitetura de firmware baseada em estruturas de trajetória, instâncias de execução e máquinas de estados;

- desenvolvimento de uma HMI para seleção de modos, envio de comandos, monitorização da comunicação série e configuração de movimentos;
- integração lógica da tosse como evento temporário sobreposto ao movimento respiratório.

6.3 Limitações

O protótipo ainda apresenta limitações importantes, desde logo porque o realismo material é limitado pelos processos de impressão disponíveis e pelos filamentos acessíveis. A solução de movimento em três eixos simplifica a dinâmica real da árvore traqueobrônquica, que inclui deformações locais, variações de calibre e interação com o broncoscópio, e a validação quantitativa dos movimentos também depende de medições experimentais futuras, nomeadamente amplitudes reais, repetibilidade, ruído, aquecimento e estabilidade da alimentação.

Outra limitação é a ausência, nesta fase, de feedback sensorial: sem sensores de contacto, força ou posição externa, o sistema depende essencialmente do comando aberto dos servos e das validações geométricas implementadas no firmware. Isto é aceitável para uma primeira prova de conceito, mas limita a simulação de resistência ao avanço do broncoscópio.

6.4 Trabalho futuro

A linha mais importante de evolução diz respeito ao modelo físico da árvore brônquica. A versão atual não simula deformação ativa do lúmen, variação de calibre ao longo do ciclo respiratório, estenoses localizadas ou resistência ao avanço do broncoscópio. Estes fenómenos têm impacto direto na perceção visual e tátil durante o procedimento real, mas exigem materiais com propriedades mecânicas controladas, atuadores distribuídos ao longo da estrutura e uma estratégia de controlo mais complexa. A integração de filamentos com diferentes durezas em zonas específicas do modelo, combinada com pequenos atuadores de deformação local, constitui a principal direção técnica para uma segunda versão.

No plano da eletrónica e da segurança, a adição de sensores de fim de curso, medição de corrente dos servos e possível feedback de posição permitiria detetar sobrecargas e posições impossíveis, aumentar a robustez do sistema e abrir caminho para simulação de resistência ao contacto com as paredes internas. Estes elementos não foram integrados nesta versão por razões de complexidade e tempo de desenvolvimento, mas a arquitetura de firmware está organizada de forma a facilitar a sua adição.

A seleção de materiais flexíveis para a árvore brônquica deve ser aprofundada em versões futuras, comparando TPU, TPE, PEBA e outros filamentos quanto à facilidade de

impressão, elasticidade, durabilidade e sensação tátil durante a navegação broncoscópica. Esta análise, que nesta versão ficou limitada a testes qualitativos, beneficiaria de ensaios mecânicos sistemáticos.

A interface gráfica no Raspberry Pi tem margem de evolução significativa: personalização de cenários, envio completo de trajetórias para o ESP32 por comunicação série, armazenamento de perfis de treino em ficheiro e monitorização mais detalhada do estado interno do firmware. A arquitetura Flask/SocketIO já utilizada facilita a adição de novas páginas e funcionalidades sem alterar a lógica de baixo nível.

Por fim, a validação com profissionais de saúde, iniciada de forma qualitativa em junho de 2026, deve ser aprofundada com uma metodologia estruturada. Esta avaliação é essencial para perceber se o movimento gerado tem valor pedagógico real, que aspetos do simulador são mais importantes para o treino de videobroncofibroscopia e que limitações da versão atual têm mais impacto na usabilidade clínica.

Bibliografia

- [1] Mohsen Davoudi e Henri G. Colt. «Bronchoscopy simulation: a brief review». Em: *Advances in Health Sciences Education* 14.2 (2008), pp. 287–296. DOI: [10.1007/s10459-007-9095-x](https://doi.org/10.1007/s10459-007-9095-x).
- [2] Cassie C. Kennedy, Fabien Maldonado e David A. Cook. «Simulation-Based Bronchoscopy Training: Systematic Review and Meta-analysis». Em: *Chest* 144.1 (2013), pp. 183–192. DOI: [10.1378/chest.12-1786](https://doi.org/10.1378/chest.12-1786).
- [3] David I. Fielding, Fabien Maldonado e Septimiu Murgu. «Achieving competency in bronchoscopy: Challenges and opportunities». Em: *Respirology* 19.4 (2014), pp. 472–482. DOI: [10.1111/resp.12279](https://doi.org/10.1111/resp.12279).
- [4] Philip Mørkeberg Nilsson et al. «Simulation in bronchoscopy: current and future perspectives». Em: *Advances in Medical Education and Practice* 8 (2017), pp. 755–760. DOI: [10.2147/AMEP.S139929](https://doi.org/10.2147/AMEP.S139929).
- [5] Mallika Gopal et al. «Bronchoscopy Simulation Training as a Tool in Medical School Education». Em: *The Annals of Thoracic Surgery* 106.1 (2018), pp. 280–286. DOI: [10.1016/j.athoracsur.2018.02.011](https://doi.org/10.1016/j.athoracsur.2018.02.011).
- [6] Pedro Barros, Bruno Santos e Ulisses Brito. «Projeto para desenvolvimento de modelo de simulação de árvore brônquica, com impressão 3D». Documento de trabalho interno, Serviço de Pneumologia, Centro Hospitalar Universitário do Algarve, cedido à equipa do projeto. 2023.
- [7] T. H. Pedersen et al. «A randomised, controlled trial evaluating a low cost, 3D-printed bronchoscopy simulator». Em: *Anaesthesia* 72.8 (2017), pp. 1005–1009. DOI: [10.1111/anae.13951](https://doi.org/10.1111/anae.13951).
- [8] Rao Fu et al. «Dynamic three-dimensional printing: The future of bronchoscopic simulation training?» Em: *Anaesthesia and Intensive Care* 51.4 (2023), pp. 274–280. DOI: [10.1177/0310057X231154015](https://doi.org/10.1177/0310057X231154015).
- [9] Peter Heinbecker. «A method for the demonstration of calibre changes in the bronchi in normal respiration». Em: *Journal of Clinical Investigation* 4.4 (1927), pp. 459–469. DOI: [10.1172/JCI100133](https://doi.org/10.1172/JCI100133).

- [10] Maxwell Ellis. «The mechanism of the rhythmic changes in the calibre of the bronchi during respiration». Em: *The Journal of Physiology* 87.3 (1936), pp. 298–309. DOI: [10.1113/jphysiol.1936.sp003408](https://doi.org/10.1113/jphysiol.1936.sp003408).
- [11] Alexander Chen et al. «The Effect of Respiratory Motion on Pulmonary Nodule Location During Electromagnetic Navigation Bronchoscopy». Em: *Chest* 147.5 (2015), pp. 1275–1281. DOI: [10.1378/chest.14-1425](https://doi.org/10.1378/chest.14-1425).
- [12] Chamindu C. Gunatilaka et al. «The effect of airway motion and breathing phase during imaging on CFD simulations of respiratory airflow». Em: *Computers in Biology and Medicine* 127 (2020), p. 104099. DOI: [10.1016/j.compbiomed.2020.104099](https://doi.org/10.1016/j.compbiomed.2020.104099).
- [13] Sissel Højsted Kronborg et al. «Four different models for simulation-based training of bronchoscopic procedures». Em: *BMC Pulmonary Medicine* 24 (2024), p. 23. DOI: [10.1186/s12890-024-02846-9](https://doi.org/10.1186/s12890-024-02846-9).
- [14] Anke Schertel, Thomas Geiser e Wolf E. Hautz. «Man or machine? Impact of tutor-guided versus simulator-guided short-time bronchoscopy training on students learning outcomes». Em: *BMC Medical Education* 21 (2021), p. 123. DOI: [10.1186/s12909-021-02526-w](https://doi.org/10.1186/s12909-021-02526-w).
- [15] Annamaria Witheridge, Gordon Ferns e Wesley Scott-Smith. «Revisiting Miller’s pyramid in medical education: the gap between traditional assessment and diagnostic reasoning». Em: *International Journal of Medical Education* 10 (2019), pp. 191–192. DOI: [10.5116/ijme.5d9b.0c37](https://doi.org/10.5116/ijme.5d9b.0c37).
- [16] Julien Pico et al. «From Realism to Learner Engagement: Rethinking Fidelity in Simulation-Based Education». Em: *JMIR Medical Education* (2026).
- [17] P. A. Baker et al. «Evaluating the ORSIM simulator for assessment of anaesthetists’ skills in flexible bronchoscopy: aspects of validity and reliability». Em: *British Journal of Anaesthesia* 117.S1 (2016), pp. i87–i91. DOI: [10.1093/bja/aew059](https://doi.org/10.1093/bja/aew059).
- [18] Amir Hossein Hadi Hosseinabadi e Septimiu E. Salcudean. «Force Sensing in Robot-assisted Keyhole Endoscopy: A Systematic Survey». Em: *The International Journal of Robotics Research* (2021).
- [19] Justin L. Garner et al. «Evaluation of a re-useable bronchoscopy biosimulator with ventilated lungs». Em: *ERJ Open Research* 5.2 (2019). DOI: [10.1183/23120541.00035-2019](https://doi.org/10.1183/23120541.00035-2019).
- [20] Martin Schulze et al. «Quality assurance of 3D-printed patient specific anatomical models: a systematic review». Em: *3D Printing in Medicine* 10 (2024), p. 9. DOI: [10.1186/s41205-024-00210-5](https://doi.org/10.1186/s41205-024-00210-5).

- [21] Carlos Aravena e Thomas R. Gildea. «Patient-specific airway stent using three-dimensional printing: a review». Em: *Annals of Translational Medicine* 11.10 (2023), p. 360. DOI: [10.21037/atm-22-2878](https://doi.org/10.21037/atm-22-2878).
- [22] Bengi Yilmaz e Bilge Yilmaz Kara. «Mathematical surface function-based design and 3D printing of airway stents». Em: *3D Printing in Medicine* 8 (2022), p. 24. DOI: [10.1186/s41205-022-00154-8](https://doi.org/10.1186/s41205-022-00154-8).
- [23] Qungang Shan et al. «Customization of stent design for treating malignant airway stenosis with the aid of three-dimensional printing». Em: *Annals of Translational Medicine* (2019).
- [24] Gregory Doucet et al. «Modelling and Manufacturing of a 3D Printed Trachea for Cricothyroidotomy Simulation». Em: *Annals of Simulation Technology* (2017).
- [25] Lily Park et al. «Increasing Access to Medical Training With Three-Dimensional Printing: Creation of an Endotracheal Intubation Model». Em: *JMIR Medical Education* (2023).
- [26] Brian Han Khai Ho et al. «Multi-material three dimensional printed models for simulation of bronchoscopy». Em: *BMC Medical Education* 19 (2019), p. 236. DOI: [10.1186/s12909-019-1677-9](https://doi.org/10.1186/s12909-019-1677-9).
- [27] Christopher T. Leba et al. «Use of a 3D Printed, Color-Coded Airway Model for Bronchoscopy Training and Anatomy Learning». Em: *Journal of 3D Printing in Medicine* 7.2 (2023). DOI: [10.2217/3dp-2023-0005](https://doi.org/10.2217/3dp-2023-0005).
- [28] Bruno Santos. «Simulação dinâmica da videobroncofibroscopia em modelo impresso 3D: introdução de movimento no modelo de árvore brônquica». Documento de trabalho interno, Serviço de Pneumologia, Centro Hospitalar Universitário do Algarve, cedido à equipa do projeto. 2026.
- [29] Milovan Savanović et al. «Quantification of Lung Tumor Motion and Optimization of Treatment». Em: *Journal of Biomedical Physics and Engineering* (2023).
- [30] Stephen Dubsky et al. «Cardiogenic Airflow in the Lung Revealed Using Synchrotron-Based Dynamic Lung Imaging». Em: *Scientific Reports* 8 (2018). DOI: [10.1038/s41598-018-22540-1](https://doi.org/10.1038/s41598-018-22540-1).
- [31] Makoto Ando e Takako Nishino. «Understanding the Mechanisms of Main Bronchial Compression in Patients with Intracardiac Anomalies». Em: *Annals of Thoracic Surgery Short Reports* 2 (2024), p. 369. DOI: [10.1016/j.atssr.2024.03.008](https://doi.org/10.1016/j.atssr.2024.03.008).
- [32] Francesco Andrani et al. «Cough, a vital reflex. Mechanisms, determinants and measurements». Em: *Acta Biomedica* 89.4 (2018), pp. 477–480. DOI: [10.23750/abm.v89i4.6182](https://doi.org/10.23750/abm.v89i4.6182).

- [33] Olusegun J. Ilegbusi. «Computational modeling of cough-induced droplets and mucosal film dynamics in the upper airway for pulmonary disease classification». Em: *Frontiers in Physiology* (2025). DOI: [10.3389/fphys.2025.1666826](https://doi.org/10.3389/fphys.2025.1666826).
- [34] Yingtian Li et al. «A Soft Robotic Balloon Endoscope for Airway Procedures». Em: *Soft Robotics* 9.5 (2022). DOI: [10.1089/soro.2020.0161](https://doi.org/10.1089/soro.2020.0161).
- [35] Plan3D. *Simulador de Broncoscopia*. URL: <https://plan3d.cl/product/simulador-de-broncoscopia/> (acedido em 06/07/2026).
- [36] TruCorp. *AirSim Bronchi*. URL: <https://trucorp.com/product/airsim-bronchi/> (acedido em 06/07/2026).
- [37] USPTO. *Teaching model of the bronchial and lungs useful for teaching the biology of those organs*. URL: <https://patents.google.com/patent/US5597310A/en> (acedido em 07/07/2026).
- [38] M. E. Giannaccini et al. «Respiratory Simulator for Robotic Respiratory Tract Treatments». Em: *2017 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. 2017. DOI: [10.1109/ROBIO.2017.8324764](https://doi.org/10.1109/ROBIO.2017.8324764).
- [39] Markus A. Horvath et al. «An organosynthetic soft robotic respiratory simulator». Em: *APL Bioengineering* 4 (2020).
- [40] Rachael Pare et al. «Electro-mechanical Lung Simulator Using Polymer and Organic Human Lung Equivalents for Realistic Breathing Simulation (xPULM)». Em: *Scientific Reports* (2019). DOI: [10.1038/s41598-019-56176-6](https://doi.org/10.1038/s41598-019-56176-6).
- [41] Jina Chang, Tae-Suk Suh e Dong-Soo Lee. «Development of a deformable lung phantom for the evaluation of deformable registration». Em: *Journal of Applied Clinical Medical Physics* (2010).
- [42] Rong-Heng Zhao et al. «Precision-Controlled Bionic Lung Simulator for Dynamic Respiration Simulation». Em: *Bioengineering (Basel)* (2025).
- [43] USPTO. *Servo lung simulator and related control method*. URL: <https://patents.google.com/patent/US5975748A/en> (acedido em 07/07/2026).
- [44] CNIPA. *Chest retraction simulating device and medical patient simulator having the device*. URL: <https://patents.google.com/patent/CN101447141B/en> (acedido em 07/07/2026).
- [45] EPO. *Ensemble d'apprentissage et simulateur de torse de nourrisson pour l'apprentissage du geste de la kinésithérapie respiratoire*. URL: <https://patents.google.com/patent/EP2430628B1/fr> (acedido em 07/07/2026).
- [46] CNIPA. *Full-automatic high-simulation body position drainage simulation training model*. URL: <https://patents.google.com/patent/CN107346634B/en> (acedido em 07/07/2026).

- [47] Atsuo Takanishi et al. *WKA (Waseda-Kyoto Kagaku Airway) airway management training robot series*. URL: <https://link.springer.com/article/10.1007/s11548-008-0238-1> (acedido em 07/07/2026).
- [48] Andrea Dunn Beltran et al. *Stop Holding Your Breath: CT-Informed Gaussian Splatting for Dynamic Bronchoscopy*. 2026. arXiv: [2604.28179](https://arxiv.org/abs/2604.28179). URL: <https://arxiv.org/abs/2604.28179> (acedido em 07/07/2026).
- [49] Joshua B. Gafford et al. «A Concentric Tube Robot System for Rigid Bronchoscopy: A Feasibility Study on Central Airway Obstruction Removal». Em: *Annals of Bio-medical Engineering* 48.1 (2019), pp. 181–191. DOI: [10.1007/s10439-019-02325-x](https://doi.org/10.1007/s10439-019-02325-x).
- [50] Daniel Cíntora Morales e Daniel Octavio Delgado Vargas. «Prototipo Simulador de Broncoscopia». Trabalho Terminal, Engenharia em Biónica. Instituto Politécnico Nacional, UPIITA, 2012.
- [51] Formlabs. *Flexible 80A Resin*. URL: <https://formlabs.com/eu/products/flexible-80a-resin/> (acedido em 07/07/2026).
- [52] Inslogic. *Inslogic Flexible 70A Resin*. URL: <https://www.inslogic3d.com/products/inslogic-flexible-70a-resin> (acedido em 07/07/2026).
- [53] Liqcreate. *Is there a flexible SLA 3D-printing resin?* URL: <https://www.liqcreate.com/supportarticles/elastic-flexible-sla-resin/> (acedido em 07/07/2026).
- [54] Formlabs. *Introducing Elastic Resin: A Soft, Resilient 3D Printing Material*. URL: <https://formlabs.com/uk/blog/elastic-resin-soft-resilient-3d-printing/> (acedido em 07/07/2026).
- [55] Formlabs. *100% Silicone 3D Printing Made Accessible: Introducing Silicone 40A Resin*. URL: <https://formlabs.com/blog/silicone-3d-printing-material-silicone-40a-resin/> (acedido em 07/07/2026).
- [56] William Bailes. *Delta-Kinematics-Library*. Biblioteca Arduino de código aberto para cinemática direta e inversa de manipuladores delta. 2018. URL: <https://github.com/tinkersprojects/Delta-Kinematics-Library> (acedido em 03/07/2026).
- [57] Alex Nikanorov. *Delta robot kinematics*. Tutorial online, HyperTriangle. URL: <https://hypertriangle.com/~alex/delta-robot-tutorial/> (acedido em 03/07/2026).
- [58] isaac879. *Delta-Robot*. Repositório de código aberto; robô manipulador delta impresso em 3D controlado por Arduino Nano. 2019. URL: <https://github.com/isaac879/Delta-Robot> (acedido em 03/07/2026).

- [59] P. J. Zsombor-Murray. *Descriptive Geometric Kinematic Analysis of Clavel's "Delta" Robot*. McGill University, Centre for Intelligent Machines, 2004. URL: <https://cim.mcgill.ca/~paul/clavdelt.pdf> (acedido em 03/07/2026).
- [60] M. Hasanlu e M. Siavashi. «Optimum kinematic–dynamic performance of the reconfigurable delta robot through genetic algorithm optimization». Em: *Robotic Systems and Applications* 5.1 (2025). DOI: [10.21595/rsa.2025.24731](https://doi.org/10.21595/rsa.2025.24731).
- [61] Byeong-Jun Kim et al. «A novel computer modeling and simulation technique for bronchi motion tracking in human lungs under respiration». Em: *Physical and Engineering Sciences in Medicine* 46 (2023), pp. 1741–1753. DOI: [10.1007/s13246-023-01336-2](https://doi.org/10.1007/s13246-023-01336-2).
- [62] Roberto W. Dal Negro et al. «Prevalence of tracheobronchomalacia and excessive dynamic airway collapse in bronchial asthma of different severity». Em: *Multidisciplinary Respiratory Medicine* 8 (2013), p. 32.
- [63] Jonathan E. Hiorns, Oliver E. Jensen e Bindi S. Brook. «Nonlinear Compliance Modulates Dynamic Bronchoconstriction in a Multiscale Airway Model». Em: *Biophysical Journal* 107 (2014), pp. 3030–3042.

Apêndice A

Anexos

A.1 Anexo A: Desenhos técnicos e modelo 3D

Incluir vistas do modelo, ficheiros CAD relevantes, parâmetros de impressão 3D e notas de montagem.

A.2 Anexo B: Lista de materiais

Incluir lista completa de peças, fornecedores, referências, quantidades e custos.

A.3 Anexo C: Datasheets

Incluir datasheets dos principais componentes eletrónicos, atuadores, drivers e fonte de alimentação.

A.4 Anexo D: Código e configurações

Incluir excertos relevantes de firmware, ficheiros de configuração, mapas de pinos e instruções de carregamento.

A.5 Anexo E: Protocolos de teste

Incluir grelhas de teste, formulários de avaliação, tabelas de resultados brutos e notas de calibração.